CHƯƠNG 1
TỔNG QUAN VỀ AI TẠO SINH

Sau khi học xong chương này, người học có thể:

- *Trình bày được khái niệm AI, phân biệt các nhánh AI – ML – DL – GenAI và mô tả được các cột mốc chính trong lịch sử phát triển của AI.*
- *Giải thích được nguyên lý hoạt động của AI tạo sinh: cơ chế học và tạo sinh, kiến trúc Transformer, Self-Attention, và các khái niệm token, context window, hallucination, RAG.*
- *Phân tích được vai trò của AI và AI tạo sinh trong từng giai đoạn của SDLC và nhận diện được lợi ích, thách thức khi tích hợp AI vào quy trình phát triển phần mềm.*
- *Sử dụng được các công cụ AI cơ bản (ChatGPT, Claude, GitHub Copilot) và vận dụng được các kỹ thuật prompt engineering (Zero-Shot, Few-Shot, CoT, ReAct) cho tác vụ lập trình.*

1.1 Tổng quan về Trí tuệ nhân tạo

Sau khi hoàn thành mục này, sinh viên có thể:

- *Trình bày được khái niệm Trí tuệ nhân tạo và phân biệt các nhánh chính: AI, Machine Learning, Deep Learning, Generative AI.*
- *Mô tả được các cột mốc chính trong lịch sử phát triển của AI.*
- *Nhận diện được các lĩnh vực ứng dụng tiêu biểu của AI trong thực tiễn và trong kỹ nghệ phần mềm.*

1.1.1 AI đã ở quanh ta từ bao giờ?

Hãy thử mở một công cụ tìm kiếm, gõ vài từ khóa và quan sát kết quả trả về. Bạn có để ý rằng công cụ đó không chỉ tìm những trang chứa chính xác từ bạn gõ, mà còn "đoán" được ý định tìm kiếm của bạn? Khi bạn mở một ứng dụng mạng xã hội, những bài viết xuất hiện đầu tiên gần như luôn là những nội dung bạn quan tâm. Khi bạn nhập một câu tiếng Việt vào Google Dịch, hệ thống trả về bản dịch tiếng Anh gần như tức thì với độ chính xác ngày càng cao. Đó chỉ là ba trong số hàng trăm ví dụ cho thấy Trí tuệ nhân tạo (Artificial Intelligence – AI) đã len lỏi vào cuộc sống hàng ngày từ lâu, trước cả khi cụm từ "Generative AI" hay "ChatGPT" trở nên phổ biến.

Tuy nhiên, với sự xuất hiện của các công cụ như ChatGPT (tháng 11/2022), AI đã bước qua một ngưỡng mới: từ chỗ chủ yếu nhận diện và phân loại, AI tiến tới sáng tạo – tạo ra văn bản, hình ảnh, âm thanh và mã nguồn hoàn toàn mới. ChatGPT đạt một triệu người dùng chỉ trong năm ngày, trở thành một "khoảnh khắc iPhone" của ngành AI – thời điểm công nghệ vốn đã phát triển âm thầm hàng thập kỷ bỗng trở nên trực quan và

dễ tiếp cận với tất cả mọi người.

Câu hỏi đặt ra cho chúng ta – những kỹ sư phần mềm tương lai – không phải là "AI có thay thế mình không?", mà là làm thế nào để khai thác AI như một công cụ trong suốt vòng đời phát triển phần mềm. Để trả lời câu hỏi đó, trước hết chúng ta cần hiểu rõ: AI thực sự là gì, nó đã phát triển như thế nào, và đâu là vị trí của AI tạo sinh trong bức tranh tổng thể.

1.1.2 AI là gì?

Thuật ngữ Trí tuệ nhân tạo (Artificial Intelligence – AI) được John McCarthy đặt ra vào năm 1956 tại hội thảo Dartmouth – sự kiện được coi là cột mốc khai sinh của ngành AI. McCarthy định nghĩa AI là "khoa học và kỹ thuật tạo ra các máy móc thông minh". Qua hơn sáu thập kỷ, định nghĩa này vẫn giữ nguyên giá trị cốt lõi, dù phạm vi và năng lực của AI đã mở rộng vượt bậc.

Ngày nay, AI được hiểu là một thuật ngữ bao trùm (umbrella term), chỉ mọi hệ thống máy tính có khả năng mô phỏng trí thông minh của con người: suy nghĩ, học hỏi, suy luận và thực hiện các tác vụ phức tạp. AI không phải là một công nghệ đơn lẻ, mà là một tập hợp các cách tiếp cận đa dạng – từ những hệ thống dựa trên luật (rule-based systems) đơn giản cho đến các mô hình học tự động (autonomous learning models) phức tạp.

Điều quan trọng cần nhấn mạnh: AI không phải là một công nghệ mới. Trước khi Generative AI bùng nổ vào năm 2022, AI và Machine Learning đã được ứng dụng rộng rãi trong nhiều thập kỷ ở các lĩnh vực như tài chính, y tế, sản xuất và công nghệ thông tin. Các hệ thống phát hiện gian lận thẻ tín dụng, bộ lọc thư rác (spam filter), hệ thống đề xuất sản phẩm trên các sàn thương mại điện tử – tất cả đều là những ứng dụng AI đã hoạt động từ lâu trước khi ChatGPT ra đời.

1.1.3 Các nhánh của AI: Từ truyền thống đến hiện đại

a. AI truyền thống

Trong những thập kỷ đầu, AI phát triển chủ yếu theo hướng hệ chuyên gia (expert systems) và xử lý ký hiệu (symbolic processing). Các hệ thống này hoạt động dựa trên các quy tắc được con người định nghĩa tường minh: nếu điều kiện A xảy ra thì thực hiện hành động B. Ưu điểm của cách tiếp cận này là dễ giải thích và kiểm soát, nhưng hạn chế lớn là không thể xử lý các tình huống mà lập trình viên chưa lường trước.

Các nhánh chính của AI truyền thống bao gồm:

- **Xử lý ngôn ngữ tự nhiên** (Natural Language Processing – NLP): giúp máy tính hiểu và xử lý ngôn ngữ con người, ứng dụng trong dịch máy, phân tích cảm xúc, chatbot dựa trên luật.
- **Thị giác máy tính** (Computer Vision): giúp máy tính "nhìn" và hiểu hình ảnh, ứng dụng trong nhận diện khuôn mặt, phát hiện đối tượng, xe tự hành.
- **Robotics**: kết hợp AI với cơ khí để tạo ra robot thông minh có khả năng tương tác với môi trường vật lý.
- **Hệ chuyên gia** (Expert Systems): mô phỏng khả năng ra quyết định của chuyên gia trong một lĩnh vực hẹp, ví dụ như chẩn đoán y khoa hay đánh giá rủi ro tín dụng.

b. Machine Learning – Học máy

Bước ngoặt lớn đầu tiên của AI đến với Machine Learning (ML – Học máy). Khác với AI truyền thống dựa trên luật, ML cho phép hệ thống học từ dữ liệu mà không cần được lập trình tường minh cho từng tình huống. Thay vì con người phải viết ra mọi quy tắc, hệ thống ML tự động nhận diện các mẫu (patterns) trong dữ liệu, từ đó dự đoán kết quả hoặc ra quyết định.

Có bốn hướng tiếp cận chính trong ML:

1. **Học có giám sát** (Supervised Learning): hệ thống học từ dữ liệu đã được gán nhãn (labeled data). Mỗi mẫu dữ liệu đầu vào đi kèm với một đầu ra mong muốn, và mô hình học cách ánh xạ từ đầu vào sang đầu ra. Ví dụ điển hình: phát hiện thư rác (spam detection) – mô hình được huấn luyện trên tập email đã được gán nhãn "spam" hoặc "không spam"; dự đoán giá nhà dựa trên diện tích, vị trí, số phòng.
2. **Học không giám sát** (Unsupervised Learning): hệ thống làm việc với dữ liệu không được gán nhãn, tự động khám phá các cấu trúc hoặc mẫu ẩn trong dữ liệu. Ví dụ: phân cụm khách hàng (customer segmentation) để phát hiện các nhóm khách hàng có hành vi mua sắm tương đồng; phát hiện bất thường (anomaly detection) trong giao dịch tài chính.
3. **Học bán giám sát** (Semi-Supervised Learning): kết hợp một lượng nhỏ dữ liệu có nhãn với một lượng lớn dữ liệu không nhãn. Cách tiếp cận này đặc biệt hữu ích khi việc gán nhãn dữ liệu tốn kém hoặc cần chuyên gia, như trong chẩn đoán ảnh y khoa.
4. **Học tăng cường** (Reinforcement Learning – RL): một agent học cách ra quyết định thông qua tương tác với môi trường. Agent thực hiện hành động, nhận phần thưởng (reward) nếu hành động tốt hoặc hình phạt (penalty) nếu hành động xấu, và điều chỉnh chiến lược để tối đa hóa tổng phần thưởng theo thời gian. RL được ứng dụng mạnh mẽ trong game (AlphaGo), robotics và điều khiển tự động.

c. Deep Learning – Học sâu

Deep Learning (DL – Học sâu) là một tập con của ML, sử dụng mạng nơ-ron nhân tạo (artificial neural networks) gồm nhiều tầng (layers) để trích xuất các đặc trưng phức tạp từ dữ liệu thô. Điểm khác biệt then chốt giữa DL và ML truyền thống nằm ở cách xử lý đặc trưng (features):

- Trong ML truyền thống, kỹ sư phải thủ công thiết kế và lựa chọn các đặc trưng từ dữ liệu thô trước khi đưa vào mô hình. Đây là bước đòi hỏi kiến thức chuyên ngành sâu và tốn nhiều thời gian.
- Trong DL, mạng nơ-ron tự động học các đặc trưng theo cấp bậc: các tầng đầu học các đặc trưng đơn giản (cạnh, góc trong ảnh), các tầng sâu hơn kết hợp chúng thành các khái niệm phức tạp (khuôn mặt, đối tượng).

Sự bùng nổ của DL vào thập niên 2010 được thúc đẩy bởi ba yếu tố: (1) dữ liệu lớn (big data) sẵn có từ Internet; (2) sức mạnh tính toán của GPU (Graphics Processing Unit); và (3) các cải tiến về thuật toán như hàm kích hoạt ReLU, kỹ thuật dropout để chống overfitting.

d. Generative AI – AI tạo sinh

Generative AI (GenAI – AI tạo sinh) là tập con tiên tiến nhất của AI hiện nay, tập trung vào việc tạo ra nội dung hoàn toàn mới – văn bản, hình ảnh, âm thanh, video và mã nguồn – dựa trên các mẫu học được từ dữ liệu huấn luyện. Đây chính là điểm khác biệt cốt lõi: nếu AI truyền thống và ML chủ yếu tập trung vào nhận diện và phân loại (đây có phải ảnh mèo không?), thì GenAI tập trung vào sáng tạo (hãy vẽ một bức ảnh mèo theo phong cách hoạt hình).

GenAI được vận hành bởi các kiến trúc mạnh mẽ như:

- **Mạng đối kháng tạo sinh** (Generative Adversarial Networks – GANs): hai mạng nơ-ron cạnh tranh với nhau – một mạng tạo (Generator) sinh dữ liệu giả, một mạng phân biệt (Discriminator) cố gắng phát hiện dữ liệu giả. Qua quá trình cạnh tranh này, Generator ngày càng tạo ra dữ liệu giống thật hơn.
- **Bộ tự mã hóa biến phân** (Variational Autoencoders – VAEs): học phân phối xác suất của dữ liệu và tạo ra dữ liệu mới bằng cách lấy mẫu từ không gian tiềm ẩn đã học được.
- **Mô hình khuếch tán** (Diffusion Models): thêm nhiễu vào dữ liệu và học cách khử nhiễu ngược để tạo ra dữ liệu mới – nền tảng của các công cụ tạo ảnh như Stable Diffusion, DALL-E, Midjourney.
- **Mô hình ngôn ngữ lớn** (Large Language Models – LLMs): các mô hình như GPT-4, Claude, Gemini sử dụng kiến trúc Transformer để hiểu và tạo văn bản giống con người.

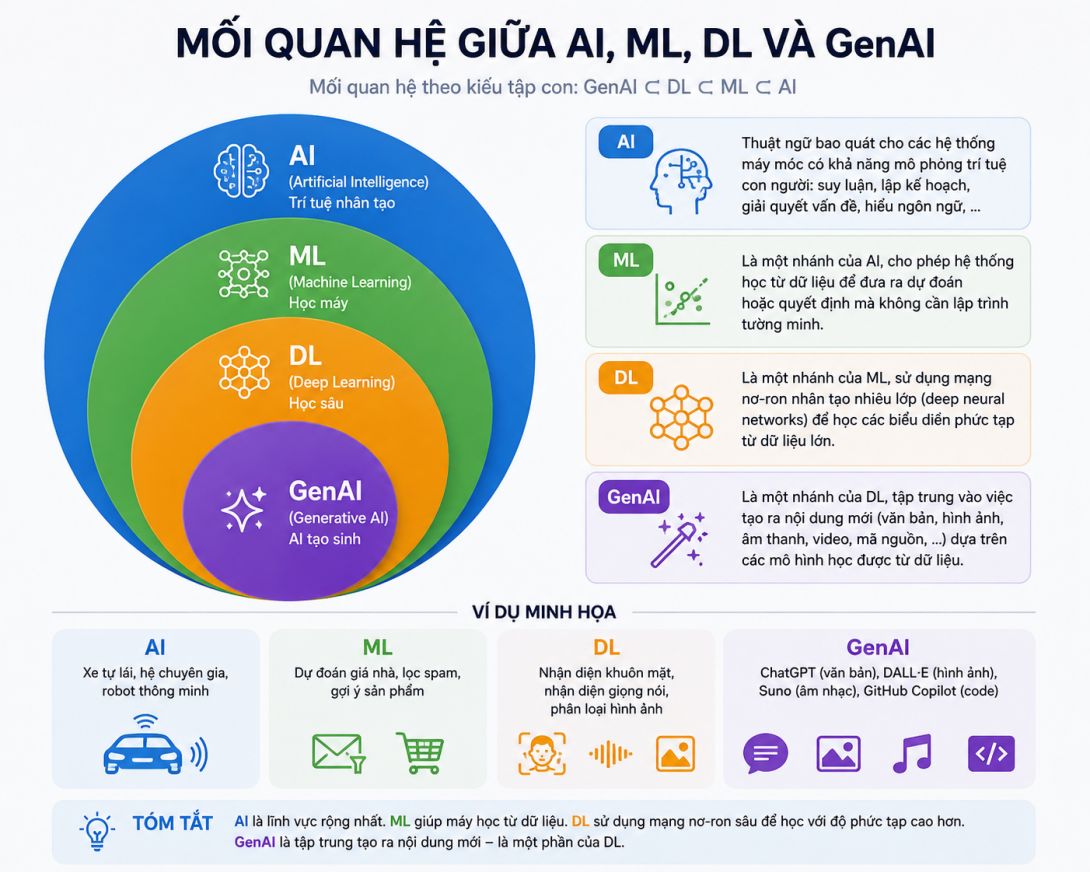
1.1.4 Mối quan hệ giữa AI, ML, DL và GenAI

Một trong những nhầm lẫn phổ biến nhất hiện nay là sử dụng các thuật ngữ AI, ML, DL và GenAI như những khái niệm đồng nghĩa hoặc có thể hoán đổi cho nhau. Thực tế, chúng có mối quan hệ phân cấp rõ ràng, như các vòng tròn đồng tâm:

Bảng 1.1 tóm tắt sự khác biệt giữa bốn khái niệm này.

Bảng 1.1: So sánh AI, ML, DL và GenAI

Tiêu chí	AI	Machine Learning	Deep Learning	Generative AI
Phạm vi	Rộng nhất: mọi hệ thống mô phỏng trí thông minh	Tập con của AI: học từ dữ liệu	Tập con của ML: dùng mạng nơ-ron sâu	Tập con của DL: tạo nội dung mới
Yêu cầu dữ liệu	Có thể không cần (hệ luật)	Cần dữ liệu có cấu trúc	Cần dữ liệu lớn (big data)	Cần dữ liệu huấn luyện khổng lồ, đa dạng
Ứng dụng điển hình	Hệ chuyên gia, NLP, robotics, computer vision	Dự đoán giá, phát hiện spam, phân cụm khách hàng	Nhận diện ảnh, giọng nói, dịch máy	Sinh văn bản, tạo ảnh, sinh code, sáng tác nhạc
Độ phức tạp	Thấp đến cao	Trung bình	Cao	Rất cao
Vai trò chính	Nhận diện & suy luận	Dự đoán & phân loại	Trích xuất đặc trưng tự động	Sáng tạo nội dung mới



Hình 1.1: Phân cấp AI – ML – DL – GenAI: mỗi vòng trong là một tập con của vòng ngoài

Một cách ghi nhớ đơn giản: AI là đại dương, ML là một biển trong đại dương đó, DL là một vịnh trong biển ML, và GenAI là một hòn đảo đặc biệt trong vịnh DL – nơi duy nhất có khả năng sáng tạo.

1.1.5 Hành trình phát triển của AI

Sự trỗi dậy của Generative AI không phải là một sự kiện đột ngột, mà là kết quả của hàng thập kỷ tiến bộ công nghệ được tích lũy. Bảng 1.2 tóm lược các cột mốc quan trọng.

Bảng 1.2: Các cột mốc chính trong lịch sử phát triển AI

Giai đoạn	Cột mốc	Ý nghĩa
1956	Hội thảo Dartmouth; John McCarthy đặt thuật ngữ "AI"	Khai sinh ngành Trí tuệ nhân tạo
Thập niên 1960–1970	Hệ chuyên gia (expert systems) ra đời; các "mùa đông AI" (AI winter) xen kẽ	AI sơ khai với hệ luật; kỳ vọng quá cao dẫn đến cắt giảm tài trợ khi không đạt được
Thập niên 1980	Máy tính cá nhân (PC) phổ biến	Dân chủ hóa sức mạnh tính toán, tạo nền tảng cho AI phát triển
Thập niên 1990	World Wide Web ra đời; ML phát triển mạnh	Dữ liệu số bùng nổ, thúc đẩy các thuật toán học máy
Thập niên 2000	Smartphone và điện toán đám mây	AI trở nên di động và dễ tiếp cận qua hạ tầng đám mây (AWS, Google Cloud, Azure)
Thập niên 2010	Deep Learning bùng nổ; GPU cho phép huấn luyện mạng nơ-ron sâu	AlexNet (2012) chiến thắng ImageNet; AI vượt trội trong thị giác máy tính, giọng nói, dịch máy
2017	Bài báo "Attention Is All You Need" giới thiệu kiến trúc Transformer	Đặt nền móng cho hầu hết LLM hiện đại (GPT, BERT, Claude)
11/2022	OpenAI ra mắt ChatGPT	"Khoảnh khắc iPhone" của AI: 1 triệu người dùng trong 5 ngày; GenAI trở thành hiện tượng toàn cầu
2023–nay	Bùng nổ hệ sinh thái GenAI: GPT-4, Claude, Gemini, Llama, Copilot	AI tạo sinh được tích hợp vào công cụ lập trình, giáo dục, y tế, sáng tạo nội dung

Lưu ý rằng lịch sử AI không phải là một đường thẳng đi lên liên tục. Ngành AI đã trải qua nhiều "mùa đông" – những giai đoạn kỳ vọng quá cao không được đáp ứng, dẫn đến cắt giảm tài trợ và mất quan tâm từ cộng đồng nghiên cứu. Tuy nhiên, mỗi mùa đông đều được tiếp nối bởi một mùa xuân mới, khi các đột phá công nghệ và sự gia tăng sức mạnh tính toán mở ra những khả năng mới.

1.1.6 AI trong kỹ nghệ phần mềm và các lĩnh vực

a. Ứng dụng AI trong đời sống và công nghiệp

AI ngày nay hiện diện trong hầu hết các lĩnh vực:

- **Y tế**: AI hỗ trợ chẩn đoán bệnh qua ảnh y khoa (X-quang, MRI), dự đoán nguy cơ bệnh, và phát triển thuốc mới. Các mô hình DL có thể phát hiện khối u trong

ảnh CT với độ chính xác tương đương hoặc vượt bác sĩ chẩn đoán hình ảnh trong một số nghiên cứu.

- **Tài chính – Ngân hàng**: AI vận hành các hệ thống phát hiện gian lận giao dịch theo thời gian thực, chấm điểm tín dụng tự động, và giao dịch thuật toán (algorithmic trading). Các chatbot AI cũng đang dần thay thế tổng đài viên trong dịch vụ khách hàng.
- **Giao thông vận tải**: Xe tự hành (autonomous vehicles) sử dụng kết hợp computer vision, sensor fusion và RL để điều hướng trong môi trường phức tạp. Các hệ thống điều phối logistics dùng AI để tối ưu tuyến đường và lịch trình giao hàng.
- **Giáo dục**: AI cá nhân hóa lộ trình học tập, tự động chấm bài và cung cấp phản hồi tức thì cho người học. Các trợ lý học tập AI có thể giải thích khái niệm phức tạp theo nhiều cách khác nhau, phù hợp với từng phong cách học.
- **Giải trí và truyền thông**: AI vận hành hệ thống đề xuất nội dung trên Netflix, YouTube, Spotify; tạo hiệu ứng đặc biệt trong phim ảnh; và gần đây là sáng tác nhạc, viết kịch bản.

b. AI trong kỹ nghệ phần mềm

Đối với sinh viên Công nghệ thông tin, ứng dụng quan trọng nhất của AI chính là trong kỹ nghệ phần mềm (software engineering). AI đang thay đổi cách lập trình viên làm việc hàng ngày:

- **Sinh mã nguồn** (Code Generation): các công cụ như GitHub Copilot, Amazon Q, ChatGPT có thể sinh hàm, lớp, thậm chí toàn bộ module từ mô tả bằng ngôn ngữ tự nhiên. Người phát triển chuyển từ việc "viết từng dòng code" sang "mô tả điều cần làm và kiểm tra kết quả".
- **Gỡ lỗi và kiểm thử** (Debugging & Testing): AI có thể phân tích stack trace, đề xuất nguyên nhân lỗi và cách sửa; tự động sinh ca kiểm thử (test cases) bao phủ các đường dẫn thực thi khác nhau; phát hiện lỗ hổng bảo mật tiềm ẩn trong mã nguồn.
- **Sinh tài liệu** (Documentation): AI tự động tạo và cập nhật tài liệu kỹ thuật từ mã nguồn và chú thích, giảm gánh nặng viết tài liệu – một trong những công việc mà lập trình viên thường ít ưu tiên nhất.
- **Phân tích yêu cầu** (Requirements Analysis): AI hỗ trợ phân tích tài liệu yêu cầu, phát hiện mâu thuẫn hoặc thiếu sót, và đề xuất cấu trúc hệ thống dựa trên mô tả chức năng.
- **Vận hành và giám sát** (Operations & Monitoring): AI phân tích log hệ thống, dự đoán sự cố trước khi xảy ra, và hỗ trợ lập kế hoạch khôi phục thảm họa (disaster recovery planning).

Một khảo sát gần đây cho thấy các lập trình viên sử dụng AI assistant tiết kiệm trung bình 30–50% thời gian cho các tác vụ lập trình thông thường, cho phép họ tập trung vào các vấn đề phức tạp hơn như thiết kế kiến trúc và tối ưu hiệu năng. Tuy nhiên, AI không thay thế được tư duy phản biện, khả năng đánh giá giải pháp và hiểu biết sâu về lĩnh vực – những kỹ năng mà giáo trình này hướng tới phát triển ở người học.

1.1.7 AI hẹp và AI tổng quát

Trước khi kết thúc, cần phân biệt hai khái niệm quan trọng:

- **AI hẹp** (Narrow AI – ANI): tất cả các hệ thống AI hiện nay, bao gồm cả những mô hình tiên tiến nhất như GPT-4 và Claude, đều là AI hẹp. Chúng được thiết kế để thực hiện xuất sắc một hoặc một vài tác vụ cụ thể, nhưng không thể tự chuyển sang lĩnh vực khác nếu không được huấn luyện lại. Một mô hình sinh văn bản không thể bỗng nhiên chơi cờ vua, và ngược lại.
- **AI tổng quát** (Artificial General Intelligence – AGI): là dạng AI lý thuyết có khả năng thực hiện bất kỳ tác vụ trí tuệ nào mà con người làm được – suy luận, lập kế hoạch, học hỏi, sáng tạo và thích nghi với mọi tình huống mới. AGI hiện vẫn đang trong giai đoạn nghiên cứu và phát triển; chưa có hệ thống nào đạt đến mức AGI. Đây là một chủ đề gây nhiều tranh luận về mặt đạo đức, an toàn và tác động xã hội.

1.1.8 Tổng kết và kết nối

Qua mục này, chúng ta đã xác lập được bức tranh tổng thể về Trí tuệ nhân tạo:

1. **AI là một lĩnh vực rộng lớn**, đã phát triển qua hơn 60 năm, không phải là công nghệ mới xuất hiện cùng với ChatGPT.
2. **AI, ML, DL và GenAI có mối quan hệ phân cấp**: AI là tập lớn nhất; ML là tập con học từ dữ liệu; DL là tập con của ML dùng mạng nơ-ron sâu; GenAI là tập con của DL tập trung vào sáng tạo nội dung mới.
3. **AI truyền thống tập trung vào nhận diện và phân loại**, GenAI tập trung vào sáng tạo – đây là sự khác biệt căn bản.
4. **AI đang thay đổi sâu sắc cách phát triển phần mềm**, từ sinh mã, kiểm thử đến vận hành. Người biết sử dụng AI sẽ có lợi thế cạnh tranh vượt trội.

Trước khi chuyển sang mục tiếp theo, hãy thử trả lời các câu hỏi sau:

1. *Phân biệt AI, ML, DL và GenAI bằng một câu cho mỗi khái niệm.*
2. *Tại sao ChatGPT (11/2022) được coi là "khoảnh khắc iPhone" của ngành AI?*
3. *Kể tên ít nhất ba ứng dụng của AI trong kỹ nghệ phần mềm mà bạn thấy có thể áp dụng ngay trong đồ án học tập của mình.*

Trong mục tiếp theo, chúng ta sẽ đi sâu vào nguyên lý hoạt động của AI tạo sinh: các mô hình nền tảng (Foundation Models), mô hình ngôn ngữ lớn (LLM), kiến trúc Transformer và cách chúng tạo ra văn bản, hình ảnh và mã nguồn.

1.2 Nguyên lý hoạt động của AI tạo sinh

Sau khi hoàn thành mục này, sinh viên có thể:

- *Giải thích được nguyên lý hoạt động cơ bản của AI tạo sinh: học từ dữ liệu và tạo nội dung mới.*
- *Phân biệt được các loại mô hình: Foundation Models, LLM, SLM, LMM.*
- *Mô tả được vai trò của kiến trúc Transformer và cơ chế self-attention.*
- *Nhận diện được các kiến trúc tạo sinh chính: GANs, VAEs và mô hình tự hồi quy.*
- *Giải thích được các khái niệm token, context window, hallucination và RAG.*

1.2.1 AI tạo sinh hoạt động như thế nào?

Trong mục trước, chúng ta đã xác định vị trí của Generative AI (GenAI) trong bức tranh AI: đây là tập con của Deep Learning, tập trung vào **sáng tạo nội dung mới**. Nhưng cụ thể, làm thế nào một cỗ máy có thể "sáng tạo"? Liệu nó có thực sự hiểu những gì nó tạo ra?

Câu trả lời ngắn gọn: GenAI không "hiểu" theo nghĩa con người. Nó học các mẫu thống kê (statistical patterns) từ dữ liệu huấn luyện, sau đó dự đoán và tạo ra nội dung mới dựa trên các mẫu đó. Đây là điểm thenh chốt cần ghi nhớ: mô hình GenAI về bản chất là một hệ thống dự đoán (predictive), không phải hệ thống hiểu biết (understanding). Nó không có ý thức, không có niềm tin, và không biết "sự thật" – nó chỉ biết những mẫu mà nó đã thấy trong dữ liệu.

1.2.2 Hai giai đoạn của GenAI: Học và Tạo sinh

Mọi mô hình GenAI, dù là tạo văn bản, hình ảnh hay âm thanh, đều hoạt động theo quy trình hai giai đoạn cơ bản:

1. **Giai đoạn Học** (Learning / Training): Mô hình được huấn luyện trên một lượng dữ liệu khổng lồ – hàng tỷ câu văn, hàng triệu bức ảnh, hàng nghìn giờ âm thanh. Trong quá trình này, mô hình không ghi nhớ từng mẫu riêng lẻ, mà **học các quy luật thống kê** ẩn sau dữ liệu: từ nào thường đi kèm với từ nào, bố cục nào tạo nên khuôn mặt, giai điệu nào theo sau giai điệu nào.
2. **Giai đoạn Tạo sinh** (Generation / Inference): Khi nhận được prompt, mô hình sử dụng những gì đã học để tạo ra nội dung mới, từng đơn vị một. Kết quả không phải là bản sao dữ liệu huấn luyện – nó là tổ hợp mới, được "lắp ráp" từ các mẫu thống kê đã học.

*Hãy liên tưởng đến cách một đứa trẻ học nói. Đứa trẻ không được dạy quy tắc ngữ pháp tường minh, mà học bằng cách nghe hàng nghìn giờ người lớn nói chuyện, rồi dần dần tự tạo ra những câu nói mới – đôi khi đúng, đôi khi sai ngữ pháp một cách "dễ thương". GenAI cũng tương tự: nó học từ dữ liệu và tạo ra nội dung mới, nhưng đôi khi cũng mắc "lỗi dễ thương" – mà trong ngành AI gọi là **hallucination** (ảo giác).*

1.2.3 Foundation Models – Mô hình nền tảng

Trung tâm của cuộc cách mạng GenAI là Foundation Models (FM – Mô hình nền tảng). Đây là các mô hình AI đa năng (general-purpose), được huấn luyện trước (pre-trained) trên tập dữ liệu cực lớn và đa dạng. Điều khiến FM khác biệt so với các mô hình AI trước đây là tính thích ứng (adaptability): một FM duy nhất có thể được tinh chỉnh (fine-tuned) để thực hiện hàng chục tác vụ khác nhau – dịch thuật, tóm tắt, sinh code, trả lời câu hỏi – mà không cần huấn luyện lại từ đầu.

Các FM hiện đại có thể xử lý nhiều loại dữ liệu khác nhau (gọi là modal): văn bản, hình ảnh, âm thanh, video. Sáu chức năng cốt lõi (core modalities) của FM bao gồm:

- **Sinh văn bản** (Text Generation) – tạo đoạn văn, bài viết, email, kịch bản.
- **Tóm tắt văn bản** (Text Summarization) – rút gọn tài liệu dài thành các ý chính.
- **Hỏi đáp** (Question Answering) – trả lời câu hỏi dựa trên kiến thức đã học.
- **Sinh mã nguồn** (Code Generation) – tạo code từ mô tả bằng ngôn ngữ tự nhiên.

- **Tạo hình ảnh** (Image Generation) – tạo ảnh từ mô tả văn bản (text-to-image).
- **Dịch thuật** (Translation) – dịch văn bản giữa các ngôn ngữ.

1.2.4 Phân loại mô hình ngôn ngữ: SLM, LLM và LMM

Dựa trên quy mô và khả năng, các mô hình ngôn ngữ được chia thành ba loại chính. Bảng 1.3 tóm tắt sự khác biệt.

Bảng 1.3: Phân loại mô hình ngôn ngữ: SLM, LLM và LMM

Tiêu chí	**SLM (Small LM)**	**LLM (Large LM)**	**LMM (Large Multi-modal Model)**
Quy mô tham số	Vài triệu đến vài tỷ (vd: Phi-3-mini: 3.8 tỷ)	Hàng chục đến hàng trăm tỷ (vd: GPT-4, Claude)	Hàng chục đến hàng trăm tỷ
Dữ liệu xử lý	Chỉ văn bản	Chủ yếu văn bản	Văn bản, hình ảnh, âm thanh, video cùng lúc
Ưu điểm	Nhẹ, chạy trên mobile & IoT; chi phí thấp	Xử lý NLP phức tạp; suy luận mạnh	Ngữ cảnh đa phương thức; ứng dụng phong phú hơn
Hạn chế	Suy luận hạn chế; dễ lỗi với tác vụ phức tạp	Yêu cầu tài nguyên tính toán lớn; chi phí vận hành cao	Độ phức tạp cao; phản hồi chậm hơn LLM thuần văn bản
Ví dụ tiêu biểu	Phi-3-mini, DistilBERT, MobileBERT	GPT-4o, Claude Sonnet, Gemini Pro, Llama 3	GPT-4o (đa phương thức), Gemini Ultra, CLIP, Flamingo

1.2.5 Kiến trúc Transformer – "Trái tim" của LLM hiện đại

Nếu Foundation Models là "bộ não" của GenAI, thì Transformer chính là "hệ thần kinh" bên trong bộ não đó. Được giới thiệu năm 2017 trong bài báo nổi tiếng *"Attention Is All You Need"*, Transformer nhanh chóng trở thành kiến trúc nền tảng cho hầu hết các LLM hiện đại.

a. Tại sao Transformer lại đặc biệt?

Trước Transformer, các mô hình xử lý ngôn ngữ chủ yếu đọc văn bản tuần tự – từng từ một, từ trái sang phải. Cách tiếp cận này có hai hạn chế lớn: (1) chậm vì không thể xử lý song song; (2) khó nắm bắt mối quan hệ giữa các từ ở xa nhau trong câu.

Transformer giải quyết cả hai vấn đề này bằng cơ chế Self-Attention (tự chú ý). Thay vì đọc tuần tự, Transformer nhìn toàn bộ câu cùng một lúc và đánh giá mức độ liên quan giữa mọi cặp từ trong câu.

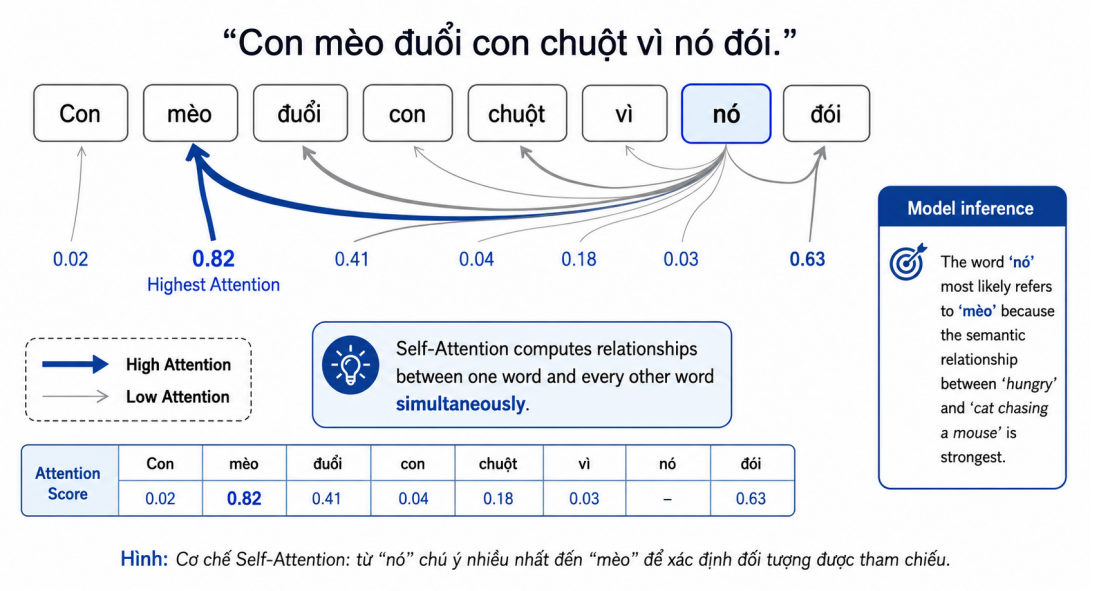
b. Cơ chế Self-Attention – Trực giác

Hãy xét câu: *"Con mèo đuổi con chuột vì nó đói."*

Câu hỏi: "nó" trong câu này ám chỉ "con mèo" hay "con chuột"? Con người dễ dàng trả lời "con mèo" vì chúng ta hiểu rằng cơn đói là động lực để săn đuổi. Nhưng làm thế nào để máy tính hiểu được điều này?

Cơ chế Self-Attention hoạt động như sau: khi xử lý từ "nó", mô hình tính toán một

điểm chú ý (attention score) giữa "nó" và mọi từ khác trong câu. Từ "mèo" sẽ nhận được điểm chú ý cao nhất vì nó có mối quan hệ ngữ nghĩa mạnh nhất với "nó" trong ngữ cảnh này. Cùng lúc, mô hình cũng đánh giá mối quan hệ giữa "đuổi" và "chuột", giữa "đói" và "mèo", v.v. – tất cả được thực hiện đồng thời trong một lần xử lý duy nhất.



Hình 1.2: Cơ chế Self-Attention: từ "nó" chú ý nhiều nhất đến "mèo" để xác định đối tượng được tham chiếu

c. Transformer trong thực tế

Về mặt khái niệm, Transformer có thể được hình dung như một người đọc có siêu năng lực: thay vì đọc từng chữ một, người này mở toang tất cả các trang sách cùng lúc, dùng bút dạ quang đánh dấu mọi mối liên hệ giữa các từ, các câu, các đoạn – rồi từ đó tổng hợp thành hiểu biết tổng thể. Chính khả năng "nhìn toàn cảnh" này giúp LLM tạo ra văn bản mạch lạc, nhất quán qua nhiều đoạn dài.

Một biến thể quan trọng là Vision Transformer (ViT), áp dụng cùng nguyên lý self-attention cho dữ liệu hình ảnh: ảnh được chia thành các mảnh nhỏ (patches), và mô hình phân tích mối quan hệ giữa chúng – tương tự cách Transformer phân tích mối quan hệ giữa các từ.

1.2.6 Các kiến trúc tạo sinh chính

Transformer là kiến trúc thống trị cho văn bản, nhưng không phải là kiến trúc duy nhất. GenAI sử dụng nhiều kiến trúc khác nhau, mỗi kiến trúc có điểm mạnh và điểm yếu riêng.

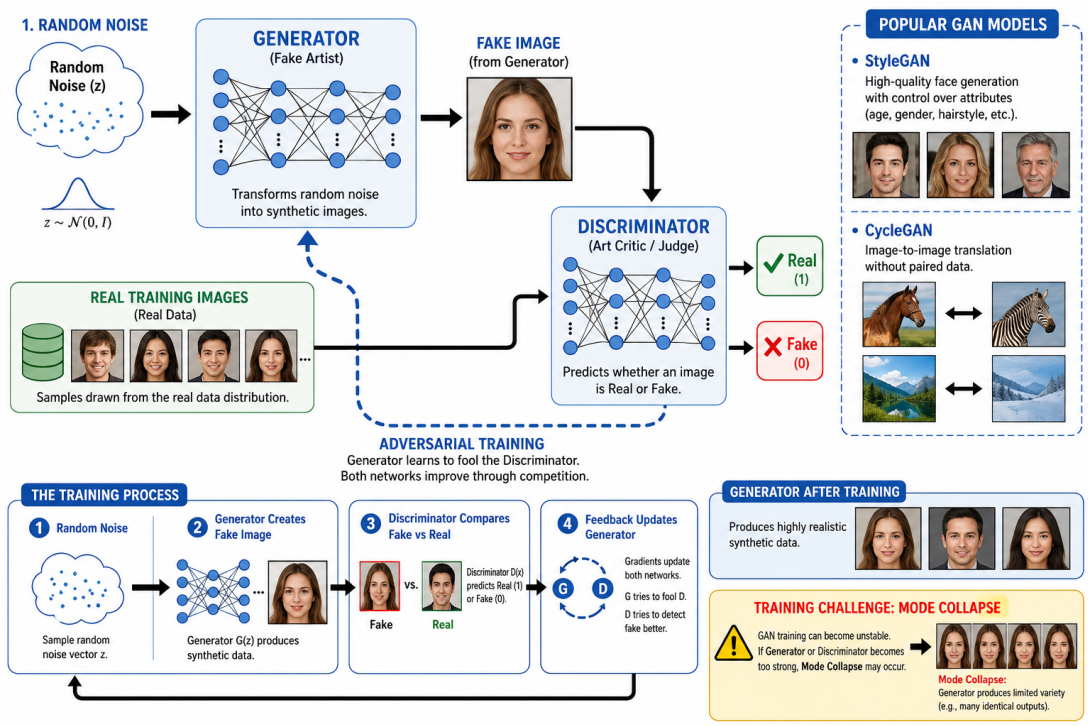
a. GANs – Mạng đối kháng tạo sinh

Generative Adversarial Networks (GANs), được Ian Goodfellow giới thiệu năm 2014, hoạt động dựa trên nguyên lý cạnh tranh. Hãy hình dung:

- **Generator** (Bộ sinh) giống như một **nghệ sĩ làm tranh giả**. Nó nhận vào nhiễu ngẫu nhiên (random noise) và cố gắng tạo ra dữ liệu trông giống thật nhất có thể.

- **Discriminator** (Bộ phân biệt) giống như một **nhà phê bình nghệ thuật**. Nó nhận cả dữ liệu thật (từ tập huấn luyện) lẫn dữ liệu giả (từ Generator), và cố gắng phân biệt đâu là thật, đâu là giả.

Hai mạng này đối kháng với nhau trong quá trình huấn luyện: Generator ngày càng tạo ra dữ liệu giống thật hơn để "lừa" Discriminator; Discriminator ngày càng tinh vi hơn để phát hiện hàng giả. Kết quả cuối cùng: Generator trở nên thành thạo đến mức dữ liệu nó tạo ra không thể phân biệt với dữ liệu thật.



Hình 1.3: Kiến trúc GANs: Generator tạo dữ liệu giả từ nhiễu; Discriminator phân biệt thật/giả; hai mạng cạnh tranh và cùng tiến bộ

GANs đặc biệt xuất sắc trong các tác vụ tạo ảnh và video, với các biến thể nổi tiếng như:

- **StyleGAN**: tạo ảnh khuôn mặt độ phân giải cao, cho phép kiểm soát các thuộc tính như tuổi, giới tính, kiểu tóc.
- **CycleGAN**: chuyển đổi ảnh giữa các phong cách mà không cần dữ liệu ghép cặp (ví dụ: biến ảnh ngựa thành ngựa vằn, ảnh mùa hè thành mùa đông).

Tuy nhiên, GANs có nhược điểm là huấn luyện không ổn định: nếu Generator hoặc Discriminator vượt trội quá xa so với bên kia, quá trình học sẽ sụp đổ (mode collapse).

b. VAEs – Bộ tự mã hóa biến phân

Variational Autoencoders (VAEs) tiếp cận vấn đề tạo sinh theo cách khác: thay vì cạnh tranh, VAEs học phân phối xác suất của dữ liệu. Cụ thể:

1. **Mã hóa** (Encoding): Dữ liệu đầu vào (ví dụ: một bức ảnh) được nén thành một điểm trong **không gian tiềm ẩn** (latent space) – một không gian có số chiều thấp hơn nhiều, biểu diễn các đặc trưng cốt lõi.

2. **Tạo sinh** (Decoding): Từ một điểm bất kỳ trong không gian tiềm ẩn, mô hình giải mã ngược lại thành dữ liệu mới. Vì không gian này liên tục và trơn tru, việc di chuyển nhẹ trong đó sẽ tạo ra các biến thể có kiểm soát của dữ liệu gốc.

Nếu GANs giống như một cuộc đấu giữa nghệ sĩ và nhà phê bình, thì VAEs giống như một **nhà điêu khắc** học cách nắn ra các bức tượng mới từ một khối đất sét bằng cách hiểu các tỉ lệ và hình dạng cơ bản.

VAEs tạo ra đầu ra đa dạng và ổn định hơn GANs, nhưng đôi khi chất lượng hình ảnh kém sắc nét hơn (hơi mờ – blurry).

c. Diffusion Models – Mô hình khuếch tán

Diffusion Models (Mô hình khuếch tán) là họ kiến trúc tạo sinh ra đời sau GANs và VAEs, hiện đang thống trị lĩnh vực tạo ảnh từ văn bản (text-to-image). Khác với GANs (cạnh tranh) và VAEs (mã hóa–giải mã), Diffusion Models hoạt động theo nguyên lý khuếch tán và khử nhiễu:

1. **Khuếch tán thuận** (Forward Diffusion): Từ một ảnh thật, mô hình thêm nhiễu (noise) từng bước một cho đến khi ảnh trở thành nhiễu thuần túy. Quá trình này được thực hiện một lần để tạo dữ liệu huấn luyện.
2. **Khử nhiễu ngược** (Reverse Diffusion): Mô hình học cách **khôi phục** ảnh từ nhiễu – bắt đầu từ nhiễu ngẫu nhiên và loại bỏ nhiễu dần qua nhiều bước, cho đến khi thu được một ảnh hoàn chỉnh. Đây chính là quá trình tạo sinh.

Điểm mạnh của Diffusion Models là chất lượng đầu ra rất cao và huấn luyện ổn định – khắc phục được cả hai nhược điểm chính của GANs (huấn luyện không ổn định) và VAEs (ảnh hơi mờ). Tuy nhiên, quá trình tạo sinh chậm hơn đáng kể vì phải thực hiện hàng chục đến hàng trăm bước khử nhiễu tuần tự. Các biến thể tiêu biểu bao gồm:

- **Stable Diffusion**: Mô hình mã nguồn mở của Stability AI, thực hiện khuếch tán trong không gian tiềm ẩn (latent space) để tăng tốc độ.
- **DALL-E 3**: Mô hình của OpenAI, tích hợp với ChatGPT để chuyển đổi mô tả văn bản thành ảnh chất lượng cao.
- **Midjourney**: Công cụ tạo ảnh thương mại nổi tiếng với phong cách nghệ thuật ấn tượng.

Bảng 1.4 so sánh bốn kiến trúc tạo sinh chính.

1.2.7 Các khái niệm then chốt khi làm việc với LLM

a. Token – Đơn vị xử lý cơ bản

LLM không đọc văn bản theo từ hay theo ký tự như con người, mà theo token. Một token có thể là một từ, một phần của từ (subword), hoặc thậm chí một ký tự đặc biệt. Ví dụ, câu *"AI tạo sinh rất thú vị"* có thể được chia thành 7 token: `AI`, `tạo`, `sinh`, `rất`, `thú`, `vị`, ..

Quá trình chia văn bản thành token được gọi là tokenization. Hầu hết các LLM sử dụng tokenizer riêng; OpenAI có một công cụ trực tuyến miễn phí (OpenAI Tokenizer) cho phép bạn dán văn bản vào và xem nó được chia thành token như thế nào.

Bảng 1.4: So sánh các kiến trúc tạo sinh chính

Tiêu chí	GANs	VAEs	Autoregressive	Diffusion
Nguyên lý	Hai mạng cạnh tranh (Generator vs Discriminator)	Học phân phối xác suất, mã hóa–giải mã	Dự đoán từng token tiếp theo dựa trên các token trước	Thêm nhiễu từng bước rồi học cách khử nhiễu ngược
Chất lượng đầu ra	Rất cao, chân thực (đặc biệt cho ảnh, video)	Đa dạng, có kiểm soát, nhưng đôi khi hơi mờ	Xuất sắc cho văn bản; đang cải thiện cho đa phương thức	Xuất sắc cho ảnh; chất lượng cao và ổn định
Độ ổn định huấn luyện	Thấp – dễ bị mode collapse	Cao – huấn luyện ổn định	Trung bình đến cao	Cao – huấn luyện ổn định
Ứng dụng chính	Tạo ảnh/video chân thực, deepfake, tăng cường dữ liệu	Tạo ảnh, nén dữ liệu, phát hiện bất thường	Sinh văn bản, dịch thuật, sinh code, hội thoại	Tạo ảnh từ văn bản, inpainting, chỉnh sửa ảnh, sinh video
Ví dụ	StyleGAN, CycleGAN	VAE gốc, Beta-VAE	GPT-4, Claude, Gemini, Llama	Stable Diffusion, DALL-E 3, Midjourney

b. Context Window – Cửa sổ ngữ cảnh

Context Window là số lượng token tối đa mà mô hình có thể "nhìn thấy" và xử lý cùng một lúc. Đây là một trong những thông số quan trọng nhất của LLM:

- **GPT-4o**: 128K token (khoảng 96.000 từ, tương đương một cuốn tiểu thuyết ngắn).
- **Claude 3.5 Sonnet**: 200K token (khoảng 150.000 từ, tương đương 400 trang văn bản).
- **Gemini 1.5 Pro**: lên đến 1M token (thử nghiệm).

Context window càng lớn, mô hình càng có thể xử lý các tài liệu dài, duy trì ngữ cảnh qua cuộc hội thoại nhiều lượt, và hiểu được các mối quan hệ phức tạp trong văn bản. Tuy nhiên, context window lớn cũng đồng nghĩa với chi phí tính toán và độ trễ cao hơn.

c. Hallucination – Khi AI "bịa đặt"

Hallucination (ảo giác AI) là hiện tượng mô hình tạo ra thông tin không chính xác, bịa đặt hoặc vô nghĩa nhưng trình bày một cách tự tin như sự thật. Ví dụ: LLM có thể bịa ra một trích dẫn khoa học không tồn tại, một vụ kiện lịch sử chưa từng xảy ra, hoặc một hàm API không có thật trong thư viện lập trình.

Nguyên nhân gốc rễ nằm ở bản chất của LLM: đây là mô hình dự đoán thống kê, không phải cơ sở tri thức. Khi không có đủ thông tin trong dữ liệu huấn luyện (hoặc khi prompt yêu cầu điều gì đó nằm ngoài phân phối đã học), mô hình vẫn phải "điền vào chỗ trống" – và nó làm điều đó theo cách có vẻ hợp lý về mặt thống kê, nhưng không đúng về mặt sự kiện.

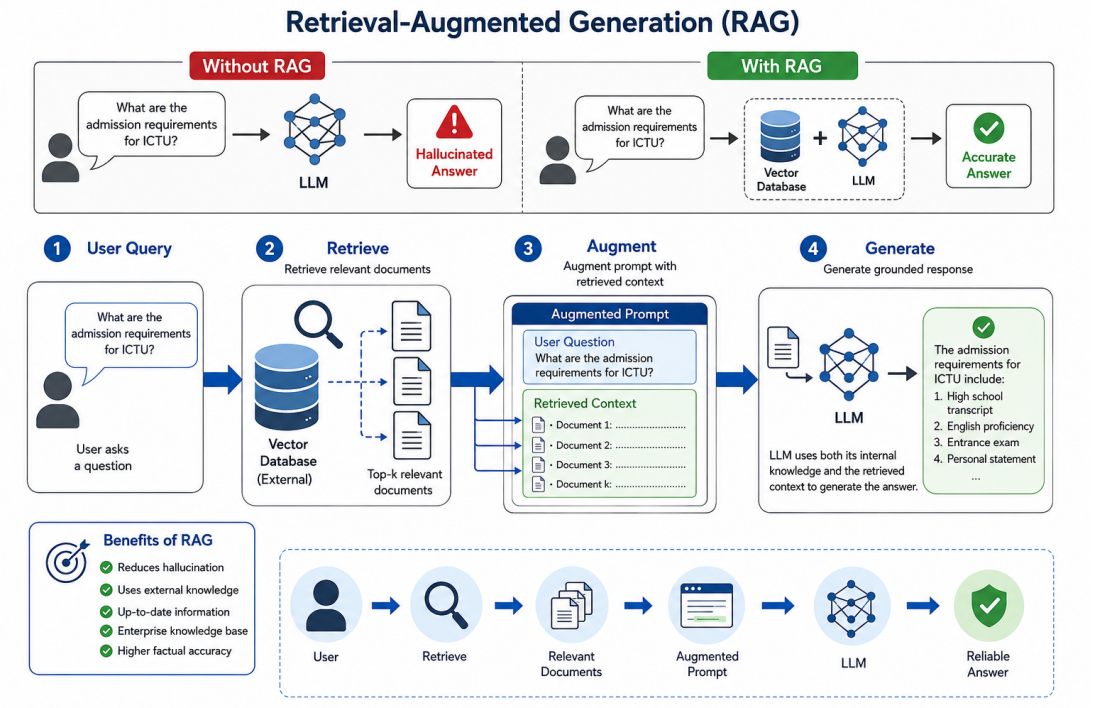
Hallucination là thách thức nghiêm trọng, đặc biệt trong các lĩnh vực nhạy cảm như y tế, pháp lý và tài chính – nơi một thông tin sai có thể gây hậu quả nghiêm trọng.

d. RAG – Giải pháp cho hallucination

Retrieval Augmented Generation (RAG – Tạo sinh tăng cường truy xuất) là phương pháp kiến trúc được phát triển để giảm thiểu hallucination và tăng độ chính xác cho LLM. Thay vì chỉ dựa vào kiến thức đã học trong quá trình huấn luyện, RAG cho phép mô hình truy xuất thông tin từ nguồn dữ liệu bên ngoài theo thời gian thực.

Quy trình hoạt động của RAG:

1. **Nhận câu hỏi** từ người dùng.
2. **Truy xuất** (Retrieve): Tìm kiếm các tài liệu, đoạn văn liên quan nhất từ cơ sở dữ liệu bên ngoài (thường là vector database chứa embedding của tài liệu).
3. **Tăng cường** (Augment): Ghép các tài liệu tìm được vào prompt gốc như ngữ cảnh bổ sung.
4. **Tạo sinh** (Generate): LLM tạo câu trả lời dựa trên cả kiến thức nội tại và thông tin vừa truy xuất được.



Hình 1.4: Quy trình RAG: truy xuất thông tin liên quan từ nguồn ngoài trước khi tạo sinh câu trả lời

RAG đặc biệt hữu ích cho các ứng dụng doanh nghiệp, nơi LLM cần trả lời chính xác dựa trên tài liệu nội bộ, chính sách công ty hoặc cơ sở kiến thức chuyên ngành – những thông tin không có trong dữ liệu huấn luyện gốc của mô hình.

1.2.8 Các Foundation Models hàng đầu hiện nay

Hệ sinh thái FM phát triển với tốc độ chóng mặt. Bảng 1.5 điểm qua một số mô hình tiêu biểu tại thời điểm đầu năm 2025.

Bảng 1.5: Một số Foundation Models tiêu biểu (đầu 2025)

Mô hình	Tổ chức	Đặc điểm nổi bật	Context Window	Mã nguồn
GPT-4o	OpenAI	Đa phương thức (text, audio, image, video); hiệu suất cao	128K token	Đóng
Claude 3.5 Sonnet	Anthropic	An toàn & đạo đức AI; suy luận mạnh; viết code xuất sắc	200K token	Đóng
Gemini 1.5 Pro	Google DeepMind	Đa phương thức; tích hợp hệ sinh thái Google	1M+ token	Đóng
Llama 3	Meta	Mã nguồn mở; cạnh tranh mô hình đóng; cộng đồng lớn	128K token	Mở
DeepSeek R1	DeepSeek	Mô hình suy luận (reasoning); giải bài toán phức tạp từng bước	128K token	Mở một phần

Lưu ý: Hệ sinh thái Foundation Models phát triển với tốc độ rất nhanh. Bảng trên phản ánh tình hình tại thời điểm biên soạn (đầu năm 2025). Khi đọc, sinh viên nên chủ động tra cứu thông tin cập nhật về các mô hình và phiên bản mới nhất.

1.2.9 Từ dự đoán đến tác tử: Agentic AI

Một xu hướng quan trọng trong GenAI hiện đại là sự chuyển dịch từ mô hình phản ứng (reactive) sang mô hình chủ động (agentic). Cách tiếp cận truyền thống (non-agentic) là: người dùng đưa prompt, mô hình trả lời một lần duy nhất – giống như hỏi một câu, nhận một câu trả lời.

Agentic AI thay đổi mô hình này: thay vì trả lời một lần, mô hình hoạt động như một tác tử (agent) – lặp đi lặp lại, tự phê bình, tự sửa và cải thiện đầu ra qua nhiều bước. Bốn mẫu hình nền tảng của agentic AI bao gồm:

1. **Reflection** (Tự phản ánh): Mô hình tạo ra đầu ra, sau đó tự kiểm tra và phê bình đầu ra đó, rồi tinh chỉnh trước khi trả về cho người dùng. Giống như một lập trình viên viết code, tự review, rồi sửa trước khi gửi đi.
2. **Tool Use** (Sử dụng công cụ): LLM tương tác với các hệ thống bên ngoài qua API – gọi hàm, truy vấn cơ sở dữ liệu, tìm kiếm web, thực thi code.
3. **Planning** (Lập kế hoạch): Phân rã một nhiệm vụ phức tạp thành các bước nhỏ hơn, sau đó thực hiện tuần tự từng bước, kiểm tra kết quả từng bước trước khi chuyển sang bước tiếp theo.
4. **Multi-agent Collaboration** (Cộng tác đa tác tử): Nhiều LLM chuyên biệt (code generation agent, testing agent, security review agent) cùng làm việc trên một nhiệm vụ, mỗi agent đảm nhận một khía cạnh chuyên môn.

Các thử nghiệm trên benchmark HumanEval của OpenAI cho thấy agentic workflows đạt độ chính xác cao hơn đáng kể so với non-agentic approach trong tác vụ tạo code – minh chứng cho tiềm năng của hướng tiếp cận này trong phát triển phần mềm.

1.2.10 Tổng kết và kết nối

Qua mục này, chúng ta đã khám phá "bên trong" của AI tạo sinh:

1. **GenAI hoạt động theo hai giai đoạn**: Học các mẫu thống kê từ dữ liệu khổng lồ, sau đó tạo ra nội dung mới bằng cách dự đoán. Nó **không hiểu** theo nghĩa con người – nó dự đoán.
2. **Foundation Models** là các mô hình đa năng, huấn luyện trước, có thể thích ứng cho nhiều tác vụ. LLM, SLM, LMM là các biến thể với quy mô và khả năng khác nhau.
3. **Transformer và Self-Attention** là "trái tim" của LLM hiện đại, cho phép mô hình nắm bắt mối quan hệ giữa mọi phần tử trong đầu vào cùng một lúc.
4. **GANs, VAEs, Autoregressive Models và Diffusion Models** là bốn kiến trúc tạo sinh chính, mỗi kiến trúc có cơ chế và thế mạnh riêng.
5. **Token, Context Window và Hallucination** là các khái niệm cốt lõi mà mọi kỹ sư phần mềm làm việc với LLM cần nắm vững.
6. **RAG** là giải pháp thực tiễn cho vấn đề hallucination, kết hợp sức mạnh của LLM với khả năng truy xuất thông tin từ nguồn ngoài.

Trước khi chuyển sang mục tiếp theo, hãy thử:

1. *Giải thích cho một người bạn không chuyên: "LLM tạo ra câu trả lời như thế nào?" bằng không quá 5 câu.*
2. *Phân biệt GANs, VAEs và Diffusion Models: mỗi loại phù hợp với bài toán nào?*
3. *Token là gì? Tại sao context window quan trọng khi chọn LLM cho một dự án?*
4. *Kể tên ít nhất hai kỹ thuật giúp giảm hallucination trong ứng dụng LLM.*

Trong mục tiếp theo, chúng ta sẽ chuyển từ "AI hoạt động như thế nào" sang "AI đang được ứng dụng ra sao trong kỹ nghệ phần mềm" – phân tích vai trò cụ thể của GenAI trong từng giai đoạn của SDLC.

1.3 Vai trò của AI trong kỹ nghệ phần mềm

Sau khi hoàn thành mục này, sinh viên có thể:

- *Phân tích được vai trò của AI tạo sinh trong từng giai đoạn của vòng đời phát triển phần mềm (SDLC).*
- *Nhận diện được các công cụ AI chính và cách chúng hỗ trợ lập trình viên.*
- *Đánh giá được lợi ích, thách thức và giới hạn khi tích hợp AI vào quy trình phát triển.*

1.3.1 AI đang thay đổi cách chúng ta xây dựng phần mềm

Nếu hai mục trước trả lời câu hỏi "AI là gì?" và "AI hoạt động như thế nào?", thì mục này trả lời câu hỏi quan trọng nhất đối với một kỹ sư phần mềm tương lai: "AI giúp ích gì cho công việc của tôi?"

Hãy hình dung một ngày làm việc điển hình của lập trình viên: đọc ticket, phân

tích yêu cầu, viết code, gỡ lỗi, viết test, cập nhật tài liệu, review code của đồng nghiệp, triển khai, theo dõi log. Trong số đó, bao nhiêu thời gian thực sự dành cho tư duy thiết kế và giải quyết vấn đề – những hoạt động tạo ra giá trị cao nhất? Và bao nhiêu thời gian dành cho các tác vụ lặp đi lặp lại như viết boilerplate code, sửa lỗi cú pháp, định dạng tài liệu?

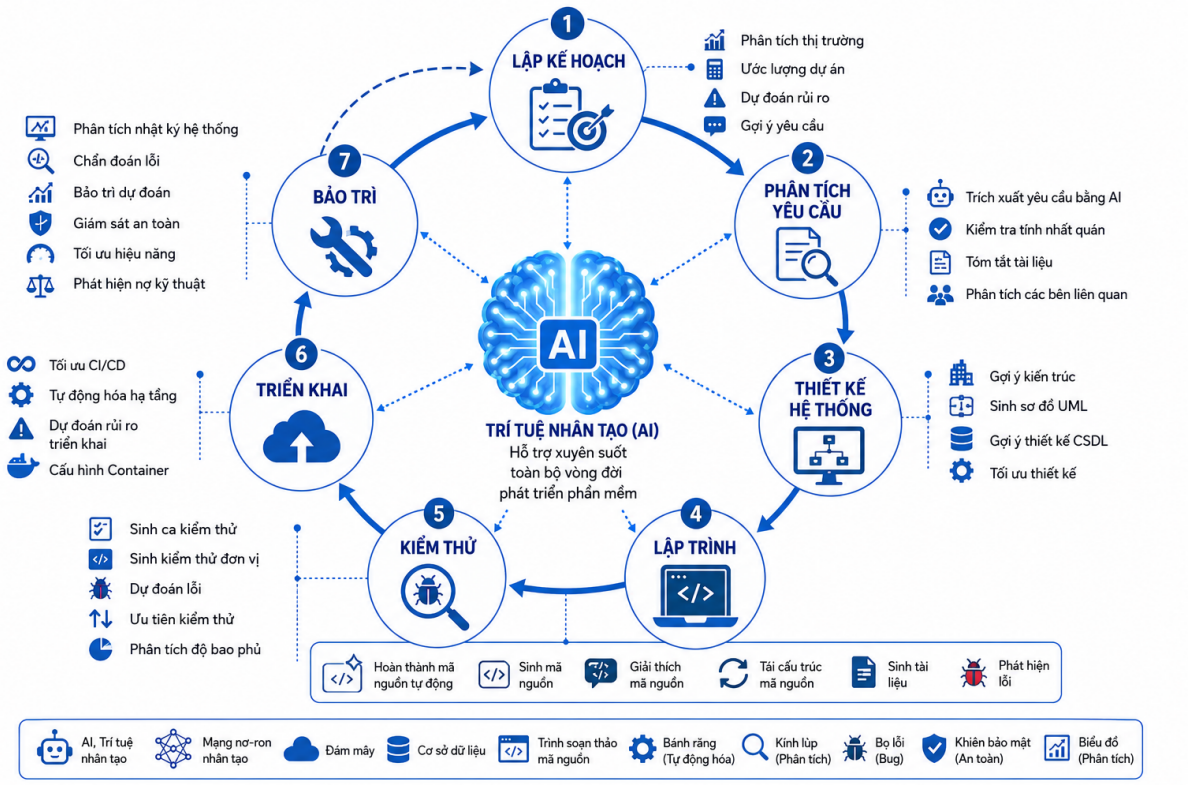
AI tạo sinh đang thay đổi cán cân này một cách căn bản. Những con số từ các công ty công nghệ hàng đầu cho thấy tác động là thực tế và đo lường được:

- **Google**: Hơn 25% code mới hiện được tạo bởi AI.
- **Amazon**: Di chuyển hàng chục nghìn ứng dụng Java 8/11 lên Java 17 với sự trợ giúp của Amazon Q Developer – công việc lẽ ra cần hơn 4.500 năm phát triển thủ công, tiết kiệm 260 triệu USD mỗi năm.
- **Meta**: CodeCompose giảm hơn 20% code lặp lại (repetitive coding).
- **Microsoft**: GitHub Copilot tăng tốc độ viết code lên 30–40%.

Nhưng GenAI không chỉ đơn thuần là "sinh code nhanh hơn". Tác động của nó trải rộng trên toàn bộ vòng đời phát triển phần mềm (Software Development Lifecycle – SDLC): từ phút đầu tiên đọc yêu cầu, đến lúc hệ thống vận hành trong production và được bảo trì nhiều năm sau đó.

1.3.2 AI trong từng giai đoạn của SDLC

Hình 1.5 tổng quan về sự hiện diện của AI trong từng giai đoạn của SDLC.



Hình 1.5: AI hiện diện trong toàn bộ vòng đời phát triển phần mềm, từ lập kế hoạch đến bảo trì

a. Lập kế hoạch và phân tích yêu cầu

Trước khi viết dòng code đầu tiên, AI đã có thể hỗ trợ:

- **Phân tích yêu cầu**: AI đọc tài liệu đặc tả, phát hiện mâu thuẫn hoặc yêu cầu thiếu rõ ràng, đề xuất làm rõ trước khi bắt đầu phát triển.
- **Phân rã công việc**: Từ một mô tả tính năng lớn, AI có thể đề xuất phân rã thành các user stories nhỏ hơn, ước tính độ phức tạp và gợi ý thứ tự ưu tiên.
- **Đề xuất công nghệ**: Dựa trên yêu cầu dự án, AI phân tích và đề xuất technology stack, kiến trúc mẫu (architectural patterns) và dịch vụ phù hợp. Ví dụ: Amazon Q Developer có thể gợi ý nên dùng dịch vụ AWS nào cho một ứng dụng web cụ thể.

Điều này đặc biệt có giá trị với các nhóm nhỏ hoặc lập trình viên mới vào nghề: AI đóng vai trò như một "kiến trúc sư dự phòng", giúp phát hiện sớm các vấn đề trước khi chúng trở thành lỗi tốn kém.

b. Phát triển và sinh mã nguồn

Đây là lĩnh vực được biết đến nhiều nhất và cũng là nơi AI tạo ra tác động tức thì nhất:

- **Sinh mã từ mô tả** (Code Generation): Từ một mô tả bằng ngôn ngữ tự nhiên – ví dụ: *"viết hàm Python upload file lên Amazon S3, có xử lý lỗi và retry"* – AI tạo ra code hoàn chỉnh, đúng cú pháp, kèm xử lý ngoại lệ.
- **Tự động hoàn thành code** (Code Completion): Khi lập trình viên gõ, AI đề xuất đoạn code tiếp theo dựa trên ngữ cảnh – không chỉ là gợi ý từ khóa như autocomplete truyền thống, mà là cả khối logic như vòng lặp, hàm xử lý, hoặc cấu trúc try-catch phù hợp. Điều này giúp lập trình viên tập trung vào logic cốt lõi thay vì cú pháp.
- **Giải thích code**: AI có thể đọc một hàm phức tạp và giải thích từng bước nó làm gì – vô cùng hữu ích khi onboarding vào dự án mới hoặc khi kế thừa codebase cũ không có tài liệu.
- **Dịch code giữa các ngôn ngữ**: AI có thể dịch một đoạn code từ Python sang JavaScript, từ Java sang Kotlin, hoặc hiện đại hóa code COBOL mainframe lên Java.

Lưu ý quan trọng: Code do AI tạo ra không phải lúc nào cũng đúng, an toàn hoặc tối ưu. Lập trình viên vẫn phải đọc hiểu, kiểm tra và chịu trách nhiệm về mọi dòng code được đưa vào sản phẩm. AI là công cụ tăng cường (augmentation tool), không phải người thay thế.

c. Kiểm thử và gỡ lỗi

Kiểm thử phần mềm – một trong những hoạt động tốn nhiều thời gian nhất trong SDLC – đang được AI biến đổi mạnh mẽ:

- **Tự động sinh test case**: AI phân tích code và tự động tạo các ca kiểm thử toàn diện, bao gồm cả **edge cases** (trường hợp biên) và **validation scenarios** – những tình huống mà lập trình viên thường bỏ sót. Điều này cải thiện đáng kể độ phủ kiểm thử (test coverage).

- **Phát hiện lỗi và lỗ hổng bảo mật**: AI phân tích các đường dẫn thực thi (code paths), truy vết bất thường trong logic, và gắn cờ các điểm có nguy cơ: SQL injection, XSS, lỗi tràn bộ đệm, điều kiện đua (race condition). Công cụ như Snyk DeepCode AI hoạt động theo thời gian thực, cảnh báo ngay khi lập trình viên gõ code.
- **Gỡ lỗi thông minh**: Thay vì đọc stack trace dài và đoán nguyên nhân, lập trình viên có thể đưa stack trace vào AI và nhận được phân tích về nguyên nhân gốc rễ kèm đề xuất cách sửa. AI thậm chí có thể đề xuất mã sửa lỗi cụ thể, để lập trình viên xem xét và áp dụng.
- **Tái cấu trúc code** (Code Refactoring): AI tự động đề xuất cải thiện cấu trúc code để tăng khả năng đọc, bảo trì và hiệu suất, mà không làm thay đổi hành vi bên ngoài. Điều này đặc biệt hữu ích khi xử lý **nợ kỹ thuật** (technical debt) trong các codebase lâu năm.

d. Tài liệu và chia sẻ kiến thức

Một trong những "nỗi đau" lớn nhất của lập trình viên là viết tài liệu. AI giải quyết vấn đề này từ hai hướng:

- **Tự động sinh tài liệu từ code**: AI phân tích cấu trúc code và tự động tạo mô tả cho hàm, lớp, module – bao gồm tham số đầu vào, giá trị trả về, ngoại lệ và ví dụ sử dụng. Có thể áp dụng cho cả code mới viết và code hiện có chưa có tài liệu.
- **Sinh tài liệu thiết kế và API**: Từ code hoặc từ mô tả, AI tạo ra tài liệu API, sơ đồ kiến trúc, và hướng dẫn tích hợp – giúp quá trình onboarding thành viên mới và bàn giao dự án nhanh hơn đáng kể.

e. CI/CD và triển khai

Trong môi trường DevOps hiện đại, AI đang được tích hợp sâu vào pipeline CI/CD:

- **Dự đoán lỗi triển khai**: Trước khi deploy, AI phân tích thay đổi và dự đoán khả năng xảy ra lỗi, đề xuất kiểm tra bổ sung hoặc chiến lược rollback.
- **Tự động hóa kiểm tra bảo mật**: AI quét mã nguồn và cấu hình triển khai để phát hiện lỗ hổng bảo mật, cấu hình sai (misconfiguration) và tuân thủ chính sách.
- **Phát hiện bất thường**: AI giám sát log xác thực, lưu lượng mạng và hành vi hệ thống để phát hiện tấn công hoặc xâm nhập bất thường.

f. Vận hành và bảo trì

Ngay cả sau khi phần mềm đã đi vào hoạt động, AI vẫn tiếp tục đóng vai trò quan trọng:

- **Phân tích nguyên nhân gốc rễ** (Root Cause Analysis – RCA): Khi xảy ra sự cố, AI phân tích log từ nhiều dịch vụ, xác định module gây lỗi và đề xuất hướng khắc phục – rút ngắn thời gian chẩn đoán từ hàng giờ xuống còn vài phút.
- **Lập kế hoạch khôi phục thảm họa** (Disaster Recovery Planning): AI phân tích kiến trúc hệ thống và đề xuất kế hoạch khôi phục phù hợp với mức RTO (Recovery Time Objective) và RPO (Recovery Point Objective) yêu cầu.
- **Tự động hóa tác vụ vận hành**: Từ đổi tên file hàng loạt, trích xuất dữ liệu từ log, đến viết script bảo trì định kỳ – AI xử lý các tác vụ sysadmin lặp đi lặp lại một cách nhanh chóng.

1.3.3 Tám lĩnh vực tác động chính

Tổng kết lại, GenAI tạo ra tác động trên tám lĩnh vực chính trong phát triển phần mềm, được tóm tắt trong Bảng 1.6.

Bảng 1.6: Tám lĩnh vực tác động chính của GenAI trong SDLC

Lĩnh vực	Mô tả	Ví dụ cụ thể
Tăng tốc phát triển (Accelerated Development)	Giảm thời gian viết code, tự động hóa tác vụ lặp	Sinh REST API endpoint từ mô tả; tự động hoàn thành code trong IDE
Nâng cao chất lượng code (Enhanced Code Quality)	Phát hiện lỗi sớm, đề xuất cải thiện cấu trúc	AI phát hiện null pointer, memory leak trước khi code được commit
Tự động hóa kiểm thử (Automated Testing)	Sinh test case, tăng độ phủ kiểm thử	Tạo unit test cho mọi nhánh logic trong một hàm
Tài liệu thông minh (Intelligent Documentation)	Tự động tạo và cập nhật tài liệu từ code	Sinh API docs, comment hàm, hướng dẫn sử dụng
Thu hẹp khoảng cách kỹ thuật – phi kỹ thuật	Product manager có thể diễn đạt yêu cầu bằng ngôn ngữ tự nhiên; AI dịch thành đặc tả kỹ thuật	PM mô tả tính năng; AI đề xuất cấu trúc dữ liệu và API tương ứng
Thúc đẩy sáng tạo (Enabling Innovation)	AI đề xuất giải pháp thay thế, cách tiếp cận mới	AI gợi ý 3 cách khác nhau để giải cùng một bài toán kèm trade-off
Mở rộng quy mô (Scaling Development)	Hỗ trợ onboarding nhanh, duy trì chất lượng khi đội ngũ mở rộng	AI giải thích codebase, trả lời câu hỏi ngữ cảnh cho thành viên mới
Quyết định dựa trên dữ liệu (Data-Driven Decisions)	Phân tích metrics, log để đề xuất cải tiến	AI phân tích hiệu suất và đề xuất tối ưu hóa cụ thể

1.3.4 Con người vẫn là trung tâm

Sau khi điểm qua tất cả những khả năng ấn tượng của AI, một câu hỏi tự nhiên xuất hiện: **vậy lập trình viên còn làm gì?**

Câu trả lời là: **lập trình viên làm những việc quan trọng hơn**. Khi AI đảm nhận các tác vụ lặp đi lặp lại, con người được giải phóng để tập trung vào:

- **Thiết kế kiến trúc**: Quyết định cấu trúc tổng thể của hệ thống – điều đòi hỏi hiểu biết sâu về lĩnh vực, kinh nghiệm thực tế và khả năng cân bằng các yếu tố đánh đổi (trade-off) mà AI hiện chưa làm được.
- **Ra quyết định chiến lược**: Chọn công nghệ nào, ưu tiên tính năng nào, đánh đổi giữa tốc độ và chất lượng ra sao – đây là những quyết định cần phán đoán kinh doanh và hiểu biết ngữ cảnh tổ chức.
- **Kiểm soát chất lượng**: AI có thể đề xuất, nhưng con người phải **xác thực** (validate). Mọi code do AI tạo ra cần được review, kiểm thử và phê duyệt bởi lập trình viên trước khi đến tay người dùng.
- **Đạo đức và trách nhiệm**: AI không chịu trách nhiệm về code nó tạo ra. Lập trình

viên chịu trách nhiệm về sản phẩm cuối cùng – bao gồm cả việc đảm bảo không có bias, không vi phạm bản quyền, và tuân thủ các quy định về bảo mật dữ liệu.

1.3.5 Thách thức khi tích hợp AI vào SDLC

Bên cạnh những lợi ích to lớn, việc tích hợp AI vào quy trình phát triển cũng đặt ra những thách thức cần được nhận diện và quản lý:

1. **Kiểm soát chất lượng đầu ra**: Code do AI sinh ra có thể chứa lỗi tinh vi, lỗ hổng bảo mật hoặc pattern không tối ưu. Nếu lập trình viên quá phụ thuộc và không kiểm tra kỹ, chất lượng sản phẩm sẽ giảm sút.
2. **Bảo mật dữ liệu**: Khi gửi code hoặc tài liệu dự án lên dịch vụ AI đám mây, dữ liệu có thể rời khỏi mạng nội bộ. Cần sử dụng enterprise plan hoặc self-hosted instance cho các dự án nhạy cảm, và tuân thủ các tiêu chuẩn như GDPR, HIPAA, SOC 2.
3. **Cân bằng chi phí – lợi ích**: API AI có chi phí tính theo token; việc gọi AI liên tục trong pipeline CI/CD có thể tạo ra chi phí đáng kể. Cần đánh giá ROI (Return on Investment) cho từng điểm tích hợp AI.
4. **Kỹ năng và đào tạo**: Sử dụng AI hiệu quả đòi hỏi kỹ năng mới – prompt engineering, đánh giá đầu ra AI, tích hợp API. Tổ chức cần đầu tư đào tạo để đội ngũ khai thác tối đa công cụ AI.
5. **Rủi ro về bản quyền và tuân thủ**: Code do AI tạo ra có thể vô tình sao chép từ mã nguồn mở có bản quyền hạn chế. Cần có quy trình kiểm tra bản quyền cho code do AI sinh ra trước khi đưa vào sản phẩm thương mại.

1.3.6 Chiến lược tích hợp AI từng bước

Thay vì cố gắng tích hợp AI vào mọi khâu cùng một lúc, các tổ chức thành công thường áp dụng chiến lược **từng bước, có đo lường**:

1. **Bắt đầu từ điểm đau nhất**: Xác định giai đoạn SDLC nào tốn nhiều thời gian nhất hoặc có tỉ lệ lỗi cao nhất, thử nghiệm AI ở đó trước.
2. **Đo lường trước và sau**: Thiết lập chỉ số cụ thể (thời gian hoàn thành task, số lỗi phát hiện, thời gian onboarding) trước khi tích hợp AI; đo lại sau khi tích hợp để đánh giá hiệu quả thực tế.
3. **Thu thập phản hồi liên tục**: Lập trình viên là người dùng cuối của công cụ AI; phản hồi của họ quyết định công cụ nào thực sự hữu ích và công cụ nào gây cản trở.
4. **Mở rộng dần**: Sau khi thành công ở một điểm, nhân rộng mô hình sang các giai đoạn SDLC khác, luôn duy trì vòng phản hồi và điều chỉnh.

1.3.7 Tổng kết và kết nối

AI tạo sinh không phải là "cơn sốt" nhất thời trong giới lập trình – nó đang **tái định hình cách phần mềm được xây dựng** ở mọi giai đoạn:

1. **Tác động toàn diện**: AI không chỉ sinh code nhanh hơn; nó hỗ trợ từ lập kế hoạch, phân tích yêu cầu, phát triển, kiểm thử, tài liệu, triển khai đến vận hành.
2. **Con số không biết nói dối**: Google, Amazon, Meta đã công bố những con số ấn tượng về mức tăng năng suất nhờ AI – minh chứng cho giá trị thực tiễn ở quy

mô doanh nghiệp.

3. **Con người là trung tâm**: AI là công cụ tăng cường, không phải người thay thế. Lập trình viên vẫn chịu trách nhiệm cuối cùng về kiến trúc, chất lượng, bảo mật và đạo đức của sản phẩm.
4. **Tích hợp thông minh, không tích hợp ồ ạt**: Chiến lược từng bước, có đo lường, dựa trên phản hồi thực tế sẽ mang lại kết quả bền vững hơn là chạy theo xu hướng.

Tự kiểm tra:

1. *Kể tên ít nhất 5 giai đoạn của SDLC và mô tả cách AI hỗ trợ trong từng giai đoạn.*
2. *Một lập trình viên mới vào nghề nên bắt đầu sử dụng AI như thế nào để mang lại lợi ích cao nhất? Đề xuất 3 tác vụ cụ thể.*
3. *Nêu 2 rủi ro khi tích hợp AI vào SDLC và cách giảm thiểu từng rủi ro đó.*
4. *Giải thích tại sao "con người vẫn là trung tâm" ngay cả khi AI có thể sinh code nhanh hơn lập trình viên.*

Trong mục tiếp theo – mục thực hành cuối cùng của chương này – chúng ta sẽ trực tiếp sử dụng các công cụ AI cơ bản: ChatGPT, GitHub Copilot và các AI-powered IDE để trải nghiệm những gì vừa thảo luận.

1.4 Bài thực hành 1

1.4.1 Thực hành với công cụ AI cơ bản

Sau khi hoàn thành bài thực hành này, sinh viên có thể:

- *Đăng ký và sử dụng được ít nhất một công cụ AI chatbot miễn phí (ChatGPT, Claude, hoặc Gemini).*
- *Viết được prompt hiệu quả cho các tác vụ lập trình cơ bản: sinh code, giải thích code, gỡ lỗi.*
- *Sử dụng được GitHub Copilot hoặc công cụ AI coding assistant tương đương trong IDE.*
- *Nhận diện và tránh được các rủi ro cơ bản khi sử dụng AI trong lập trình.*

Chuẩn bị trước buổi học:

- *Đăng ký tài khoản ChatGPT miễn phí tại* `https://chat.openai.com` *(hoặc Claude tại* `https://claude.ai`*).*
- *Cài đặt VS Code và extension GitHub Copilot (nếu có điều kiện đăng ký Copilot).*
- *Đảm bảo máy tính có kết nối Internet ổn định.*

a. Tổng quan về các công cụ AI cho lập trình viên

Trước khi bắt đầu thực hành, hãy hình dung bức tranh tổng thể. Các công cụ AI dành cho lập trình viên hiện nay có thể chia thành ba nhóm chính, được tóm tắt trong Bảng 1.7.

Bảng 1.7: Phân loại công cụ AI cho lập trình viên

Nhóm	Đặc điểm	Ví dụ	Phù hợp cho
Chatbot AI đa năng	Tương tác qua hội thoại; dùng cho nhiều tác vụ: sinh code, giải thích, gỡ lỗi, viết tài liệu, học tập	ChatGPT (OpenAI), Claude (Anthropic), Gemini (Google)	Học tập, khám phá, tác vụ đa dạng không gắn chặt vào IDE
Công cụ sinh code trong IDE	Tích hợp trực tiếp vào IDE; đề xuất code theo thời gian thực dựa trên ngữ cảnh file đang làm việc	GitHub Copilot, Amazon Q Developer, Tabnine, Codeium	Viết code hàng ngày, tự động hoàn thành, sinh boilerplate
AI-Assisted IDE	Môi trường phát triển được thiết kế lại từ đầu với AI là thành phần trung tâm	Cursor AI, Replit (Ghostwriter), AWS Cloud9 với Q Developer	Phát triển dự án toàn diện với AI đồng hành xuyên suốt

Nguyên tắc vàng: *Không có công cụ nào là "tốt nhất" cho mọi tác vụ. Một lập trình viên hiệu quả thường kết hợp:*

- *Dùng **chatbot AI** (ChatGPT/Claude) để phân tích bài toán, lên ý tưởng giải pháp, học công nghệ mới.*
- *Dùng **Copilot/Q Developer** để tăng tốc viết code trong IDE.*
- *Dùng **AI-assisted IDE** khi cần AI tham gia sâu vào toàn bộ quy trình phát triển.*

b. Làm quen với Chatbot AI

Mục tiêu Làm quen với giao diện chatbot AI, viết prompt đầu tiên, và hiểu cách AI phản hồi.

Các bước thực hiện **Bước 1 – Đăng nhập:** Mở trình duyệt, truy cập `https://chat.openai.com` (hoặc `https://claude.ai`) và đăng nhập bằng tài khoản đã đăng ký.

Bước 2 – Prompt đầu tiên: Nhập prompt sau vào ô chat và quan sát kết quả:

```
Viết một hàm Python tính tổng các số chẵn trong một danh sách.
Yêu cầu:
- Tên hàm: tinh_tong_chan
- Có docstring giải thích bằng tiếng Việt
- Có kiểm tra đầu vào (input validation)
- Kèm theo 3 ví dụ sử dụng
```

Bước 3 – Yêu cầu giải thích: Sau khi nhận được code, tiếp tục hỏi:

```
Giải thích từng dòng trong hàm trên và cho biết tại sao lại
dùng cách kiểm tra đầu vào như vậy.
```

Bước 4 – Yêu cầu cải tiến: Yêu cầu AI cải tiến code:

```
Có cách nào để tối ưu hàm trên cho danh sách rất lớn (hàng triệu
phần tử) không? Nếu có, hãy viết lại hàm và giải thích sự khác biệt.
```

Quan sát và thảo luận:

- *AI trả lời nhanh hay chậm? Có đúng yêu cầu không?*
- *Code do AI sinh ra có cần sửa gì trước khi chạy được không?*
- *Khi bạn yêu cầu giải thích, AI có giải thích đúng và dễ hiểu không?*
- *So sánh phản hồi giữa các sinh viên dùng cùng một prompt – có khác nhau không?*

c. *Kỹ thuật Prompt Engineering cơ bản*

Prompt Engineering là kỹ năng **thiết kế lời nhắc** để đạt được đầu ra chất lượng cao từ AI. Đây là một trong những kỹ năng quan trọng nhất khi làm việc với AI tạo sinh.

Năm thành phần của một prompt hiệu quả Một prompt tốt thường chứa tối đa năm thành phần (không phải prompt nào cũng cần tất cả), được minh họa trong Bảng 1.8.

Bảng 1.8: Năm thành phần của một prompt hiệu quả

Thành phần	Vai trò	Ví dụ
Instructions (Hướng dẫn)	Nói rõ AI cần làm gì	*"Viết một hàm Python ..."*
Context (Ngữ cảnh)	Cung cấp background và tình huống	*"Tôi đang xây dựng ứng dụng web bán hàng, cần xử lý giỏ hàng..."*
Input Data / Constraints (Dữ liệu / Ràng buộc)	Đưa dữ liệu cụ thể hoặc giới hạn	*"Dùng Python 3.10+, không dùng thư viện ngoài, thời gian chạy dưới 1 giây..."*
Examples (Ví dụ)	Cho AI thấy mẫu đầu vào – đầu ra mong muốn	*"Ví dụ: input [1,2,3,4] → output 6; input [5,6,7] → output 6..."*
Output Format (Định dạng đầu ra)	Quy định cấu trúc và hình thức kết quả	*"Trả về JSON với các trường: function_name, code, complexity, test_cases..."*

Bốn kỹ thuật Prompt cơ bản

1. **Zero-Shot Prompting** – Yêu cầu AI thực hiện tác vụ **không kèm ví dụ**. Phù hợp với tác vụ đơn giản, rõ ràng.

```
Dịch đoạn code Python sau sang JavaScript:
def greet(name):
    return f"Hello, {name}!"
```

2. **Few-Shot Prompting** – Cung cấp **vài ví dụ** để AI hiểu pattern mong muốn. Cực kỳ hiệu quả khi bạn cần đầu ra theo định dạng cụ thể.

```
Chuyển các mô tả sau thành docstring Python. Ví dụ:
- Mô tả: "hàm kiểm tra số nguyên tố"
  Docstring: "Kiểm tra xem một số nguyên dương có phải là số
  nguyên tố hay không."
- Mô tả: "hàm tính trung bình các số dương trong danh sách"
  Docstring: "Tính trung bình cộng của các số dương trong một
  danh sách. Bỏ qua các số không dương."

Bây giờ hãy chuyển mô tả sau:
- Mô tả: "hàm kiểm tra email hợp lệ"
```

3. **Chain of Thought (CoT) Prompting** – Hướng dẫn AI **suy nghĩ từng bước**. Đặc biệt hiệu quả với bài toán logic, giải thuật và gỡ lỗi.

```
Tìm lỗi trong đoạn code sau. Hãy suy nghĩ từng bước:
1. Đọc code và xác định mục đích của nó
2. Chạy thử bằng tay với một vài input
3. Xác định dòng nào gây ra lỗi
4. Giải thích tại sao dòng đó sai
5. Đề xuất cách sửa

def binary_search(arr, target):
    left, right = 0, len(arr)
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid
    return -1
```

4. **ReAct Prompting** – Kết hợp **Suy luận + Hành động**. Dùng khi cần AI vừa phân tích vừa thực hiện.

```
Tôi cần xây dựng một API REST để quản lý danh sách công việc (todo
↪ list).
```

```
Hãy phân tích và đề xuất từng bước:
1. Xác định các endpoint cần thiết
2. Với mỗi endpoint, mô tả HTTP method, URL pattern,
   request/response format
3. Đề xuất cấu trúc dữ liệu để lưu trữ
4. Viết code Python Flask cho endpoint "tạo công việc mới"
```

Thực hành Prompt Engineering

1. **So sánh kỹ thuật prompt:** Cùng một yêu cầu "viết hàm sắp xếp danh sách từ điển theo một khóa", hãy thử:
   - Dùng Zero-Shot: chỉ nêu yêu cầu, không thêm gì.
   - Dùng Few-Shot: thêm một ví dụ mẫu.
   - Dùng CoT: yêu cầu AI giải thích từng bước trước khi viết.

   So sánh chất lượng code nhận được từ ba cách.
2. **Sửa prompt xấu:** Prompt sau đây quá mơ hồ:

   ```
   Viết cho tôi cái app quản lý sinh viên đi.
   ```

   Hãy viết lại prompt này thành một prompt tốt, có đủ Instructions, Context, Constraints và Output Format.
3. **Prompt cho gỡ lỗi:** Tự viết một đoạn code Python ngắn có chứa lỗi (ví dụ: lỗi off-by-one, lỗi logic, hoặc lỗi kiểu dữ liệu). Dùng CoT prompting để yêu cầu AI tìm và sửa lỗi. Đánh giá: AI có tìm đúng lỗi không? Có giải thích rõ ràng không?

d. Sử dụng AI Coding Assistant trong IDE

Mục tiêu Trải nghiệm cách AI hỗ trợ viết code trực tiếp trong môi trường phát triển.

Lựa chọn công cụ Tùy theo điều kiện, chọn một trong các phương án sau (xếp theo độ ưu tiên):

1. **GitHub Copilot** (nếu có tài khoản đăng ký): Cài đặt extension trong VS Code.
2. **Codeium** (miễn phí cho cá nhân): Cài đặt extension Codeium trong VS Code – giải pháp thay thế Copilot miễn phí, chất lượng tốt.
3. **ChatGPT/Claude** qua trình duyệt (nếu không cài được extension): Dùng copy-paste code giữa IDE và chatbot.

Thực hành với AI Coding Assistant Thực hiện lần lượt các thao tác sau trong VS Code (hoặc IDE bạn chọn):

Thao tác 1 – Sinh code từ comment: Tạo một file Python mới, viết comment mô tả chức năng, rồi để AI sinh code:

```
# Hàm đọc file CSV chứa dữ liệu điểm sinh viên,
# tính điểm trung bình, và trả về danh sách sinh viên
# có điểm trung bình >= 5.0
```

```
# Cột trong CSV: MSSV, HoTen, DiemGK, DiemCK
```

Copilot/Codeium sẽ tự động đề xuất phần thân hàm. Nhấn `Tab` để chấp nhận.

Thao tác 2 – Tự động hoàn thành: Viết dở một vòng lặp xử lý dữ liệu và quan sát AI gợi ý phần còn lại:

```
students = [
    {"name": "An", "scores": [7, 8, 9]},
    {"name": "Binh", "scores": [5, 6, 7]},
    {"name": "Chi", "scores": [8, 9, 10]},
]

# In ra tên và điểm trung bình của từng sinh viên
```

Thao tác 3 – Sinh test case: Viết một hàm đơn giản, sau đó yêu cầu AI sinh unit test:

```
def is_valid_password(password: str) -> bool:
    """Kiểm tra mật khẩu hợp lệ: ít nhất 8 ký tự,
    có ít nhất 1 chữ hoa, 1 chữ thường, 1 số."""
    if len(password) < 8:
        return False
    has_upper = any(c.isupper() for c in password)
    has_lower = any(c.islower() for c in password)
    has_digit = any(c.isdigit() for c in password)
    return has_upper and has_lower and has_digit

# TODO: Dùng AI để sinh unit test cho hàm trên
# Prompt gợi ý: "Viết 5 unit test cases cho hàm is_valid_password
# bằng Python unittest, bao gồm cả edge cases"
```

Thao tác 4 – Giải thích code lạ: Dán một đoạn code phức tạp (có thể lấy từ thư viện open-source hoặc bài tập cũ) vào chatbot và yêu cầu giải thích. So sánh mức độ hiểu trước và sau khi AI giải thích.

e. So sánh và lựa chọn công cụ

Mục tiêu Hiểu được điểm mạnh, điểm yếu của từng công cụ và biết cách chọn công cụ phù hợp cho từng tác vụ.

Thực hành so sánh

1. **Cùng một prompt, nhiều công cụ:** Dùng cùng prompt "*Viết một REST API bằng Python Flask để quản lý thư viện sách: thêm sách, mượn sách, trả sách, tìm sách theo tên*" và gửi cho ít nhất hai công cụ khác nhau (ví dụ: ChatGPT và Claude, hoặc ChatGPT và Gemini).

2. **So sánh kết quả** theo các tiêu chí trong Bảng 1.9 (sinh viên tự điền kết quả).

Bảng 1.9: So sánh công cụ AI – Phiếu thực hành (sinh viên tự điền)

Tiêu chí	**Công cụ 1: ……**	**Công cụ 2: ……**
Code có chạy được ngay không?		
Có xử lý lỗi / validation không?		
Có giải thích code không?		
Chất lượng code (sạch, rõ ràng, đúng chuẩn)		
Tốc độ phản hồi		
Có đề xuất cải tiến gì thêm không?		
Ví dụ cách điền: *Công cụ 1: ChatGPT – Có chạy được ngay? "Có, nhưng cần sửa import." | Xử lý lỗi? "Có try-catch đầy đủ." | Giải thích? "Có, rất chi tiết từng dòng." | Chất lượng? "Code sạch, đặt tên rõ ràng." | Tốc độ? "Khoảng 3 giây." | Đề xuất cải tiến? "Gợi ý dùng async/await."*		

3. **Thảo luận nhóm:** Mỗi nhóm 2–3 sinh viên thảo luận và trả lời:
   - Khi nào nên dùng chatbot AI thay vì coding assistant trong IDE?
   - Có nên trả phí để dùng phiên bản Pro của các công cụ AI không? Khi nào thì đáng?
   - Công cụ AI nào phù hợp nhất cho: (a) học lập trình, (b) làm đồ án, (c) đi làm doanh nghiệp?

f. Sử dụng AI an toàn và có trách nhiệm

Sức mạnh của AI đi kèm với trách nhiệm. Trước khi kết thúc buổi thực hành, hãy nắm vững các nguyên tắc sau:

1. **Không chia sẻ dữ liệu nhạy cảm**: Tuyệt đối không gửi mã nguồn dự án thật của công ty, dữ liệu khách hàng, khóa API, mật khẩu, hoặc thông tin cá nhân lên các dịch vụ AI công cộng. Nếu cần dùng AI cho dự án doanh nghiệp, sử dụng enterprise plan hoặc self-hosted instance.
2. **Luôn kiểm tra đầu ra AI**: Code do AI tạo ra có thể chứa lỗi, lỗ hổng bảo mật, hoặc vi phạm bản quyền. Mọi code do AI sinh ra phải được **đọc hiểu, kiểm thử và review** trước khi đưa vào sản phẩm.
3. **Minh bạch về việc sử dụng AI**: Trong môi trường học tập, ghi rõ phần nào do AI hỗ trợ. Trong môi trường doanh nghiệp, tuân thủ chính sách của công ty về việc sử dụng AI.
4. **Kiểm tra bias và tính chính xác**: AI có thể tạo ra thông tin sai lệch (hallucination) hoặc phản ánh định kiến (bias) từ dữ liệu huấn luyện. Đặc biệt thận trọng khi dùng AI cho các lĩnh vực nhạy cảm như tài chính, y tế, pháp lý.
5. **Tuân thủ bản quyền**: Code do AI tạo ra có thể vô tình sao chép từ mã nguồn có bản quyền. Sử dụng công cụ kiểm tra bản quyền nếu code sẽ được dùng trong sản phẩm thương mại.

Tình huống thảo luận: *Bạn là thực tập sinh tại một công ty phần mềm. Sếp giao cho bạn sửa một lỗi trong module xử lý thanh toán. Bạn định copy-paste code đó vào ChatGPT để AI giúp tìm lỗi. Điều này có an toàn không? Nếu không, bạn nên làm gì thay thế?*
Gợi ý: Thay vì gửi nguyên code thật, hãy tạo một phiên bản code mẫu (toy example) có cùng pattern lỗi nhưng không chứa dữ liệu thật hoặc logic nghiệp vụ nhạy cảm.

g. Tổng kết bài thực hành

Qua hai tiết thực hành, bạn đã:

1. **Làm quen với Chatbot AI**: Viết prompt, nhận code, yêu cầu giải thích và cải tiến.
2. **Nắm được 4 kỹ thuật Prompt Engineering**: Zero-Shot, Few-Shot, Chain of Thought, ReAct.
3. **Sử dụng AI Coding Assistant**: Sinh code từ comment, tự động hoàn thành, sinh test case.
4. **So sánh và lựa chọn công cụ**: Hiểu được không có công cụ nào là "tốt nhất tuyệt đối".
5. **Hiểu nguyên tắc an toàn**: Không chia sẻ dữ liệu nhạy cảm, luôn kiểm tra đầu ra AI.

Bài tập về nhà:

1. *Dùng AI chatbot viết một script Python nhỏ giúp ích cho việc học tập của bạn (ví dụ: script đổi tên file hàng loạt, script tổng hợp điểm từ nhiều file, script tạo flashcard học từ vựng). Yêu cầu:*
    - *Sử dụng ít nhất 2 kỹ thuật prompt khác nhau trong quá trình phát triển.*
    - *Ghi lại các prompt đã dùng và giải thích lý do chọn kỹ thuật đó.*
    - *Code phải chạy được và có xử lý lỗi cơ bản.*
2. *Viết một báo cáo ngắn (1–2 trang A4) so sánh trải nghiệm sử dụng ít nhất hai công cụ AI khác nhau cho cùng một tác vụ lập trình. Nêu rõ công cụ nào phù hợp hơn cho phong cách làm việc của bạn và lý do.*

1.4.2 Phân tích yêu cầu hệ thống bằng AI

Xây dựng hệ thống Quản lý bán hàng (thực hiện xuyên suốt từ Bài 1 đến Bài 9).
Quy định chung (áp dụng cho tất cả các bài thực hành):

- *Sinh viên có thể lựa chọn ngôn ngữ lập trình, framework, hệ quản trị cơ sở dữ liệu, công cụ thiết kế và nền tảng triển khai phù hợp.*
- *Có thể sử dụng một hoặc nhiều công cụ AI tạo sinh. Mọi kết quả do AI đề xuất phải được kiểm tra, hiệu chỉnh và chịu trách nhiệm bởi sinh viên.*
- *Không cung cấp cho công cụ AI công cộng dữ liệu cá nhân, thông tin bí mật, mật khẩu, khóa API hoặc dữ liệu chưa được phép chia sẻ.*
- *Mỗi nhóm duy trì **Nhật ký sử dụng AI** gồm: ngày thực hiện, nhiệm vụ, công cụ, prompt, phản hồi chính, nội dung đã sửa và lý do sửa (xem mẫu tại Phụ lục).*
- *Đầu ra của bài trước được dùng làm đầu vào cho bài sau. Khi thay đổi yêu cầu hoặc thiết kế, sinh viên phải cập nhật các tài liệu liên quan.*

Sinh viên sử dụng một hoặc nhiều công cụ AI tạo sinh để phân tích bài toán xây dựng hệ thống Quản lý bán hàng. Từ yêu cầu ban đầu của cửa hàng, sinh viên khai thác AI nhằm xác định các đối tượng tham gia, chức năng chính, yêu cầu chức năng, yêu cầu phi chức năng và các kịch bản sử dụng. Trên cơ sở kết quả do AI đề xuất, sinh viên phân tích, đánh giá, chỉnh sửa và hoàn thiện tài liệu đặc tả yêu cầu phần mềm, sơ đồ Use Case và User Story. Kết quả của bài thực hành là bộ tài liệu phân tích yêu cầu làm cơ sở cho toàn bộ quá trình phát triển hệ thống ở các bài tiếp theo.

a. Mục tiêu

- Thu thập, phân loại và làm rõ yêu cầu của hệ thống Quản lý bán hàng.
- Biết đặt prompt để AI đóng vai trò trợ lý phân tích nghiệp vụ.
- Xây dựng danh sách tác nhân, Use Case, User Story và tiêu chí chấp nhận.
- Nhận diện yêu cầu mơ hồ, mâu thuẫn, thiếu sót và rủi ro.
- Hoàn thiện tài liệu yêu cầu bằng đánh giá của con người thay vì sao chép nguyên kết quả AI.

b. Chuẩn bị

- Mô tả ban đầu của cửa hàng: loại hàng hóa, quy mô, người dùng, quy trình bán hàng và các báo cáo mong muốn.
- Một công cụ AI tạo sinh có khả năng xử lý văn bản.
- Công cụ soạn thảo tài liệu và vẽ sơ đồ tùy chọn.
- Kiến thức cơ bản về yêu cầu chức năng, phi chức năng, Use Case và User Story.

c. Hướng dẫn thực hiện

Bước 1: Xây dựng bối cảnh bài toán

- Mô tả loại cửa hàng, số lượng nhân viên, kênh bán hàng, phương thức thanh toán và phạm vi hệ thống.
- Nêu rõ những nội dung chưa làm trong phiên bản đầu, ví dụ: kế toán chuyên sâu, thương mại điện tử hoặc tích hợp vận chuyển.

- Tạo ít nhất 10 câu hỏi cần phỏng vấn chủ cửa hàng.

Bước 2: Sử dụng AI để khám phá yêu cầu

- Cung cấp bối cảnh đã ẩn danh cho AI.
- Yêu cầu AI xác định tác nhân, mục tiêu của từng tác nhân, quy trình chính và các ngoại lệ.
- Yêu cầu AI đặt câu hỏi bổ sung thay vì tự giả định thông tin còn thiếu.
- Lưu prompt và phản hồi vào Nhật ký sử dụng AI.

Bước 3: Phân loại và ưu tiên yêu cầu

- Lập bảng yêu cầu gồm: mã, mô tả, loại yêu cầu, nguồn, mức ưu tiên, tiêu chí xác minh và ghi chú.
- Phân loại thành yêu cầu chức năng, phi chức năng, dữ liệu, giao diện và ràng buộc.
- Ưu tiên theo bốn mức: bắt buộc, nên có, có thể có, chưa thực hiện.
- Kiểm tra các yêu cầu có thể đo lường được, đặc biệt với hiệu năng, bảo mật và khả dụng.

Bước 4: Xây dựng Use Case

- Xác định tối thiểu các tác nhân: quản trị viên, nhân viên bán hàng, quản lý và khách hàng nếu phù hợp.
- Vẽ sơ đồ Use Case thể hiện ranh giới hệ thống và quan hệ giữa các trường hợp sử dụng.
- Viết đặc tả chi tiết cho ít nhất 5 Use Case quan trọng, gồm: mục tiêu, tác nhân, tiền điều kiện, kích hoạt, luồng chính, luồng thay thế, hậu điều kiện và quy tắc nghiệp vụ.

Bước 5: Xây dựng User Story

- Viết User Story theo mẫu: "Là [vai trò], tôi muốn [mục tiêu] để [lợi ích]".
- Mỗi User Story có tiêu chí chấp nhận rõ ràng theo dạng Given–When–Then hoặc danh sách điều kiện kiểm tra được.
- Sắp xếp User Story thành backlog cho phiên bản đầu tiên.

Bước 6: Phản biện và hoàn thiện

- Yêu cầu AI tìm điểm mơ hồ, trùng lặp, mâu thuẫn và trường hợp biên.
- Đối chiếu từng nhận xét với bối cảnh thực tế; chấp nhận hoặc bác bỏ có giải thích.
- Lập ma trận truy vết giữa mục tiêu nghiệp vụ, yêu cầu, Use Case và User Story.
- Chốt phạm vi phiên bản đầu tiên của hệ thống.

d. Prompt mẫu

Prompt 1 – Khám phá yêu cầu:

"Bạn là chuyên viên phân tích nghiệp vụ. Hãy phân tích bối cảnh hệ thống Quản lý bán hàng sau: [mô tả bối cảnh]. Trước tiên, hãy liệt kê những thông

tin còn thiếu và đặt tối đa 12 câu hỏi làm rõ. Không tự giả định. Sau đó đề xuất tác nhân, mục tiêu nghiệp vụ, yêu cầu chức năng, yêu cầu phi chức năng và các quy tắc nghiệp vụ dưới dạng bảng."

Prompt 2 – Phản biện đặc tả:

"Hãy đóng vai người phản biện yêu cầu phần mềm. Kiểm tra danh sách yêu cầu sau: [nội dung]. Với mỗi vấn đề, nêu mã yêu cầu, loại lỗi (mơ hồ, thiếu, mâu thuẫn, không kiểm thử được hoặc vượt phạm vi), lý do và cách sửa. Không bổ sung chức năng mới nếu chưa có căn cứ từ bối cảnh."

e. Kiểm tra và hiệu chỉnh kết quả AI

- Xác minh mọi tác nhân và chức năng đều xuất phát từ nhu cầu thực tế.
- Loại bỏ giả định không có căn cứ và các chức năng vượt phạm vi.
- Đảm bảo yêu cầu phi chức năng có chỉ số hoặc điều kiện đánh giá.
- Kiểm tra Use Case không mô tả chi tiết công nghệ khi chưa cần thiết.
- Ghi rõ nội dung nào của AI đã được sử dụng, sửa đổi hoặc bác bỏ.

f. Sản phẩm cần nộp

- Tài liệu bối cảnh, phạm vi và thuật ngữ của dự án.
- Danh sách yêu cầu đã phân loại và ưu tiên.
- Sơ đồ Use Case và ít nhất 5 đặc tả Use Case.
- Backlog User Story kèm tiêu chí chấp nhận.
- Ma trận truy vết yêu cầu.
- Nhật ký sử dụng AI của Bài 1.

g. Tiêu chí đánh giá (10 điểm)

- Bối cảnh, phạm vi và câu hỏi làm rõ: 1,5 điểm.
- Danh sách yêu cầu đầy đủ, rõ và có ưu tiên: 2,0 điểm.
- Sơ đồ và đặc tả Use Case: 2,0 điểm.
- User Story và tiêu chí chấp nhận: 1,5 điểm.
- Ma trận truy vết và tính nhất quán: 1,0 điểm.
- Khả năng phản biện, hiệu chỉnh đầu ra AI: 1,0 điểm.
- Hình thức tài liệu và Nhật ký sử dụng AI: 1,0 điểm.

Ôn tập cuối chương

Tóm tắt kiến thức cần nhớ

- **AI là lĩnh vực rộng lớn** đã phát triển hơn 60 năm, không phải công nghệ mới. AI, ML, DL và GenAI có quan hệ phân cấp chặt chẽ: AI ⊃ ML ⊃ DL ⊃ GenAI. AI truyền thống tập trung vào **nhận diện và phân loại**; GenAI tập trung vào **sáng tạo nội dung mới**.
- **GenAI hoạt động theo hai giai đoạn**: (1) Học các mẫu thống kê từ dữ liệu huấn luyện khổng lồ; (2) Dự đoán và tạo ra nội dung mới từng đơn vị một. GenAI về bản chất là hệ thống **dự đoán**, không phải hệ thống **hiểu biết** – nó không có ý thức hay niềm tin.
- **Foundation Models** là các mô hình đa năng được huấn luyện trước, có thể thích ứng cho nhiều tác vụ khác nhau. Dựa trên quy mô và khả năng, mô hình ngôn ngữ được chia thành SLM (nhẹ, chạy trên thiết bị di động), LLM (mạnh, suy luận phức tạp) và LMM (đa phương thức, xử lý đồng thời văn bản, hình ảnh, âm thanh).
- **Transformer với cơ chế Self-Attention** là "trái tim" của LLM hiện đại. Thay vì xử lý tuần tự, Self-Attention cho phép mô hình **nhìn toàn bộ đầu vào cùng lúc** và đánh giá mức độ liên quan giữa mọi cặp từ – giải quyết được hạn chế về tốc độ và khả năng nắm bắt ngữ cảnh xa của mô hình tuần tự.
- **Bốn kiến trúc tạo sinh chính**: GANs (hai mạng cạnh tranh – chất lượng cao nhưng huấn luyện không ổn định), VAEs (học phân phối xác suất – ổn định, đa dạng), Autoregressive Models như GPT (dự đoán token kế tiếp – xuất sắc cho văn bản), và Diffusion Models (khử nhiễu tuần tự – chất lượng ảnh vượt trội, huấn luyện ổn định).
- **Token** là đơn vị xử lý cơ bản của LLM; **Context Window** là số token tối đa mô hình có thể xử lý cùng lúc (từ 128K đến 1M+ token tùy mô hình). **Hallucination** là hiện tượng AI tạo thông tin sai nhưng trình bày tự tin – hệ quả của bản chất dự đoán thống kê. **RAG** (tạo sinh tăng cường truy xuất) là giải pháp giảm hallucination bằng cách truy xuất thông tin từ nguồn ngoài trước khi tạo sinh.
- **AI hiện diện trong toàn bộ SDLC**: từ lập kế hoạch, phân tích yêu cầu, phát triển, kiểm thử, tài liệu, CI/CD đến vận hành. Các con số thực tế từ Google (25% code mới), Amazon (tiết kiệm 260M USD/năm) và Microsoft (tăng 30–40% tốc độ viết code) cho thấy tác động là đo lường được.
- **Con người vẫn là trung tâm**: AI là công cụ **tăng cường**, không phải người thay thế. Lập trình viên chịu trách nhiệm cuối cùng về kiến trúc, chất lượng, bảo mật và đạo đức của sản phẩm. Prompt Engineering – kỹ năng thiết kế lời nhắc hiệu quả – là một trong những kỹ năng quan trọng nhất khi làm việc với AI tạo sinh.

Câu hỏi ôn tập

1. **[Nhận biết]** Vẽ sơ đồ thể hiện mối quan hệ phân cấp giữa AI, Machine Learning, Deep Learning và Generative AI. Nêu một câu mô tả ngắn gọn cho mỗi khái niệm.
2. **[Nhận biết]** Kể tên bốn kiến trúc tạo sinh chính được trình bày trong chương. Với mỗi kiến trúc, nêu một ứng dụng tiêu biểu.
3. **[Thông hiểu]** Trình bày hai giai đoạn hoạt động cơ bản của mô hình GenAI. Tại sao nói "GenAI không hiểu theo nghĩa con người"? Nếu một LLM trả lời đúng một câu hỏi lịch sử, đó là nhờ hiểu biết hay nhờ điều gì?

4. **[Thông hiểu]** Cơ chế Self-Attention trong Transformer giải quyết được hai hạn chế nào của các mô hình xử lý tuần tự? Dùng ví dụ câu *"Con mèo đuổi con chuột vì nó đói"* để minh họa cách Self-Attention xác định từ "nó" ám chỉ đối tượng nào.
5. **[Thông hiểu]** Phân biệt SLM, LLM và LMM. Một công ty muốn triển khai chatbot chăm sóc khách hàng trên thiết bị di động ngoại tuyến – loại mô hình nào phù hợp nhất? Giải thích lý do.
6. **[Thông hiểu]** Giải thích mối liên hệ giữa hallucination và RAG. Tại sao RAG được coi là giải pháp thực tiễn cho vấn đề hallucination trong ứng dụng doanh nghiệp?
7. **[Vận dụng]** Liệt kê ít nhất 5 giai đoạn của SDLC. Với mỗi giai đoạn, mô tả một cách cụ thể mà AI tạo sinh có thể hỗ trợ và cho một ví dụ công cụ hoặc kỹ thuật tương ứng.
8. **[Vận dụng]** Năm thành phần của một prompt hiệu quả là gì? Một sinh viên viết prompt: *"Viết cho tôi cái app quản lý sinh viên đi."* Hãy phân tích prompt này thiếu những thành phần nào và viết lại thành một prompt hoàn chỉnh có đủ ít nhất 4 trong 5 thành phần.
9. **[Phân tích]** So sánh hai chiến lược tích hợp AI vào SDLC: "tích hợp đồng loạt tất cả các khâu" và "tích hợp từng bước, có đo lường." Phân tích ưu điểm, nhược điểm và rủi ro của mỗi chiến lược. Bạn chọn chiến lược nào cho đồ án tốt nghiệp của mình? Tại sao?
10. **[Phân tích]** Một công ty phần mềm ghi nhận rằng sau khi áp dụng AI coding assistant, tốc độ viết code tăng 40% nhưng số lỗi trong production cũng tăng 15%. Hãy phân tích các nguyên nhân có thể dẫn đến nghịch lý này và đề xuất ít nhất ba biện pháp khắc phục dựa trên nội dung chương.

Bài tập thực hành

1. **[Làm theo mẫu – Nhận biết]** Sử dụng ChatGPT hoặc Claude, thiết kế một prompt có đủ 5 thành phần (Instructions, Context, Constraints, Examples, Output Format) để yêu cầu AI sinh hàm Python kiểm tra tính hợp lệ của địa chỉ email. Chụp màn hình prompt đã viết và code kết quả. Đánh dấu từng thành phần trong prompt của bạn.
2. **[Thay đổi có kiểm soát – Thông hiểu]** Trong Bài 1 của Mục 1.4, prompt mẫu yêu cầu viết hàm `tinh\_tong\_chan`. Hãy thay đổi yêu cầu thành: (a) tính tổng các số lẻ; (b) sử dụng list comprehension thay vì vòng lặp thông thường; (c) thêm yêu cầu hàm phải xử lý được danh sách rỗng. So sánh code được sinh ra với code từ prompt gốc. Viết một đoạn nhận xét ngắn (5–7 câu) về mức độ ảnh hưởng của từng thay đổi trong prompt đến code đầu ra.
3. **[Kết hợp – Vận dụng]** Chọn một giai đoạn trong SDLC mà bạn thấy AI có thể hỗ trợ nhiều nhất cho đồ án học tập hiện tại của mình. Thiết kế một prompt sử dụng kỹ thuật **Chain-of-Thought** (CoT) để yêu cầu AI thực hiện một tác vụ cụ thể trong giai đoạn đó. Yêu cầu: (a) ghi rõ bạn chọn giai đoạn nào và lý do; (b) viết prompt hoàn chỉnh; (c) chụp kết quả AI trả về; (d) nhận xét chất lượng kết quả theo 3 tiêu chí: tính chính xác, tính đầy đủ, tính khả thi.
4. **[Vận dụng mới – Phân tích]** Bạn được giao thiết kế một ứng dụng web nhỏ quản lý công việc cá nhân (to-do list). Hãy sử dụng AI chatbot để:

(a) Phân tích yêu cầu và đề xuất danh sách tính năng chính.
(b) Đề xuất cấu trúc dữ liệu (bảng/collection) phù hợp.
(c) Sinh code Python Flask cho API endpoint "tạo công việc mới" (POST `/api/tasks`).

Ghi lại toàn bộ hội thoại với AI. Sau đó, viết một đánh giá (10–15 câu) về trải nghiệm: AI đã hỗ trợ tốt ở đâu? Chỗ nào bạn phải tự điều chỉnh? Nếu làm lại, bạn sẽ thay đổi prompt như thế nào để có kết quả tốt hơn?

Câu hỏi thảo luận

1. **Hallucination và trách nhiệm**: Hallucination được xác định là một trong những thách thức lớn nhất của LLM. Ngoài RAG, hãy thảo luận ít nhất hai cách tiếp cận khác mà một đội phát triển có thể áp dụng để giảm thiểu rủi ro do hallucination gây ra trong ứng dụng thực tế. Liệu có thể loại bỏ hoàn toàn hallucination không? Tại sao?
2. **Tương lai nghề nghiệp**: Dựa trên nội dung mục "Con người vẫn là trung tâm" (Mục 1.3) và số liệu thực tế từ Google, Amazon, Microsoft, hãy thảo luận: liệu nỗi lo "AI sẽ thay thế lập trình viên" có cơ sở không? Những kỹ năng nào sẽ trở nên **quan trọng hơn** trong kỷ nguyên AI? Những kỹ năng nào sẽ **giảm giá trị**? Sinh viên năm cuối nên chuẩn bị gì để thích nghi với sự thay đổi này?
3. **Đạo đức và chính sách**: Trong bài thực hành, sinh viên được cảnh báo "không chia sẻ dữ liệu nhạy cảm lên dịch vụ AI công cộng." Tuy nhiên, ranh giới giữa "dữ liệu nhạy cảm" và "dữ liệu bình thường" không phải lúc nào cũng rõ ràng. Thảo luận: một công ty phần mềm nên xây dựng chính sách sử dụng AI nội bộ như thế nào để vừa khai thác được sức mạnh của AI, vừa bảo vệ được tài sản trí tuệ và dữ liệu khách hàng? Đề xuất ít nhất 3 điều khoản cụ thể cho chính sách đó.